

Do LLM-Generated Web Apps Behave Correctly?

A Contract-Grounded Benchmark for Behavioral Conformance

Anonymous ACL submission

Abstract

Large language models and coding agents can produce web applications that appear functional, yet surface plausibility is an unreliable proxy for behavioral correctness under interaction. We introduce ConformWeb, a contract-grounded protocol and benchmark that operationalizes public behavioral contracts to construct hidden interaction scenarios, evaluating whether agent-generated applications preserve strict conformance under dynamic execution. By decoupling hidden interaction sequences from the public behavioral scope, the protocol guarantees a fair held-out workflow evaluation without relying on undisclosed reference implementations. Evaluated across 24 validated instances scaling to 756 private scenarios and 6,433 semantic steps, only 252 of the 2,160 total execution runs achieve full conformance, yielding a strict pass rate of 11.7%. While scenario-level conformance reaches 36.7% and step-level progress averages 49.4%, this quantitative gap demonstrates that generative systems frequently achieve substantial partial progress but fail to maintain the critical behavioral invariants required for end-to-end conformance. All code and data are released to foster reproducibility at anonymous.4open.science/r/ConformWeb.

1 Introduction

Large language models and coding agents increasingly generate runnable web applications rather than isolated functions, scripts, or static pages (Lu et al., 2026; Xiao et al., 2025). Although these applications appear functional during brief demonstrations, they merely render pages, display controls, accept form inputs, and initiate familiar workflows as expected. Correctness for an interactive application is not established by a single screen or a local action. It depends on whether behavior remains coherent as actions accumulate across state, views, routes, permissions, services, and persistence (Trivedi et al., 2024; hattami et al., 2025).

Such coherence requires that submitted data seamlessly propagate to subsequent summary views, role changes strictly dictate actionable permissions, and page reloads faithfully preserve state data that the interface has claimed to save. Generated applications often fail at these boundaries while retaining a plausible surface (Jimenez et al., 2024; Zhu et al., 2025; Peng et al., 2026).

Existing evaluations provide important signals within their respective domains. For instance, code generation benchmarks evaluate functional compliance through isolated input-output mapping (Jimenez et al., 2024; Jain et al., 2025; Zhuo et al., 2025), while visual and front-end benchmarks measure static layout fidelity and structural replication (Si et al., 2025; Lu et al., 2026; Zhu et al., 2025; Xiao et al., 2025). Similarly, web and OS agents are assessed on their ability to complete task-oriented trajectories within pre-configured, stable environments (Zhou et al., 2024; Koh et al., 2024; Drouin et al., 2024; Xie et al., 2024). However, these benchmark settings typically fix at least one side of the interaction boundary, usually providing the callable interface, test harness, target page, or operating environment in advance. The behavioral failures of fully generated applications expose a fundamental shift in this unit of evaluation.

Because generated web applications shift the interaction boundary entirely, the system under test must autonomously produce the necessary controls, routing architectures, and application states by which behavior is validated. Consequently, the core evaluation challenge transitions from verifying static or isolated functional outputs to determining whether the application preserves its declared behavior across dynamic, multi-step operations. This operational vulnerability exposes a fundamental shift in the unit of evaluation, as the generative system can no longer rely on a pre-configured infrastructure to guide its execution.

Evaluating such generative systems requires

a parallel shift in measuring generalization. In traditional machine learning, evaluating models on held-out test sets has long been the foundational standard to prevent overfitting (Kohavi et al., 1995). To assess LLM-based application generation, this testing principle must be extended to verify whether a model can generalize to implied workflows, rather than merely replicating explicit sequences disclosed in the context. Without this rigorous check, generated applications easily become tightly coupled to specific demonstration paths and fracture when encountering alternative compositions of the same requirements. True behavioral evaluation must therefore conceal specific execution paths while verifying conformance within a publicly specified scope.

To address this challenge, we introduce ConformWeb, a contract-grounded protocol and benchmark designed to evaluate behavioral conformance under private interaction scenarios. Each benchmark instance is defined by a public behavioral contract that declares the interactions, observations, and obligations that may be evaluated as workflows unfold. Private evaluation scenarios are admissible only when their actions, evidence channels, and checks are strictly grounded in this public contract. By decoupling the hidden interaction sequences from the public behavioral scope, ConformWeb operationalizes a true held-out workflow testbed. A generated application is thus judged rigorously as a functional implementation of the declared contract, rather than by surface-level visual similarity or by rigid matching against an undisclosed reference implementation.

ConformWeb is instantiated across 24 instances spanning six application families and four complexity tiers, scaling to 756 scenarios over 6,433 semantic steps. To ensure programmatic validity, the benchmark consists exclusively of tasks verified to be fully attainable from the public contract alone. Across 2,160 comprehensive evaluation runs, strict behavioral conformance remains exceptionally rare, which is evidenced by a baseline full-pass rate of merely 11.7% and a frontier model group achievement of only 24.2% despite reaching 36.7% scenario-level conformance. This sharp divergence underscores our central empirical signal because generated applications successfully build requested controls and navigate substantial workflow prefixes but consistently fracture when long-horizon dependencies demand absolute state or persistence coherence.

Our contributions are summarized as follows.

- We define generated application correctness as coherent behavior across unseen workflow compositions, shifting the evaluation paradigm from local actions or visual similarity.
- We introduce ConformWeb, a contract-grounded framework that decouples public behavioral scope from hidden interaction sequences, ensuring private scenarios remain strictly grounded in the public contract.
- We release 24 instances spanning six families and four complexity tiers, complete with public contracts, multi-stage scenarios, reference implementations, and deterministic browser-executable checks.
- We evaluate 2,160 generations to reveal that full conformance remains rare, diagnosing a critical failure mode where models build plausible surfaces but fracture under stateful, long-horizon dependencies.

2 Related Work

2.1 Web agent evaluation

Web interaction benchmarks traditionally evaluate agents that act within pre-existing environments. Established frameworks and recent benchmarks focus on measuring perception, grounding, planning, and task completion across deployed interfaces (Yao et al., 2022; Deng et al., 2023; Zhou et al., 2024; Koh et al., 2024; He et al., 2024; Zheng et al., 2024; Lù et al., 2024; Drouin et al., 2024; Chezelles et al., 2024; Boisvert et al., 2024). In these settings, the website, task environment, and interaction substrate remain fixed throughout the evaluation.

ConformWeb evaluates a fundamentally different object. The generative system must produce the web application itself from a public behavioral specification, and correctness is determined by whether the generated application preserves that behavior under private browser trajectories. This formulation shifts the unit of evaluation from a single action or a graded agent trace to an entire user workflow. Furthermore, unlike automatic trajectory grading and LLM judge based evaluation (Zheng et al., 2023; Liu et al., 2023b; Lù et al., 2025), ConformWeb treats behavioral conformance as a deterministic testing problem. Private scenarios

function as executable tests strictly grounded in the public contract rather than qualitative judgments over an execution trajectory.

This perspective connects ConformWeb to specification based testing, model based testing, and conformance testing, where correctness is defined by adherence to a specification under systematically generated tests (Tretmans, 1996, 2008; Claessen and Hughes, 2000). Similarly, GUI and web testing research has long explored event sequences, interface models, crawling, and interaction driven failures (Memon et al., 2002; Memon, 2007; Mesbah et al., 2012; Hammoud et al., 2017). ConformWeb adapts these principles to the domain of LLM generated web applications. The public contract serves simultaneously as the generation specification, the boundary for admissible private tests, and the source of executable browser checks. While private scenarios can instantiate unique values and compose declared actions in unseen orders, they cannot introduce behavior absent from the public contract. This design guarantees that the hidden evaluation remains fair because the workflow sequences are concealed while the behavioral scope remains fully public.

2.2 Code and web application generation

Code generation benchmarks typically evaluate the functional correctness of isolated functions, scripts, programs, or repository patches (Chen et al., 2021; Odena et al., 2021; Hendrycks et al., 2021; Lai et al., 2023; Lu et al., 2021; Puri et al., 2021; Casano et al., 2023; Liu et al., 2023a; Jain et al., 2025; Zhuo et al., 2025; Jimenez et al., 2024; Zan et al., 2026). In parallel, interface and website generation benchmarks focus on visual reconstruction, front end layout generation, component behavior, multimodal page understanding, or page level functionality (Beltramelli, 2018; Si et al., 2025; Lin et al., 2025; Awal et al., 2025; Lu et al., 2026; Zhu et al., 2025; Xu et al., 2025). While these evaluations provide vital executable and visual signals, the unit under test is restricted to a single function, component, or disclosed task.

ConformWeb instead treats the generated artifact as a stateful, end to end web application navigated through dynamic browser trajectories. Rather than measuring surface similarity or performance on a single disclosed workflow, correctness is defined by full-layer behavioral conformance when declared obligations are composed in undisclosed private scenarios.

3 ConformWeb Protocol

ConformWeb operationalizes the public contract principle into an executable evaluation protocol where each benchmark instance comprises a product premise, a public behavioral contract, and both public and private scenarios. While public scenarios provide disclosed representative workflows, private scenarios challenge the application with held-out workflow compositions that remain strictly grounded within the same public contract. Consequently, a generated application is evaluated based on its compliance with declared behaviors rather than its proximity to an undisclosed reference implementation. By isolating surface realization from public and private scenario conformance, the protocol successfully distinguishes between superficial interface omissions, breakdowns on disclosed tasks, and deep failures that emerge only under stateful composition. Figure 1 illustrates this operational framework, and Appendix A details the underlying contract language, compiler, and browser runner.

3.1 Contracts and Private Scenarios

The contract is the object that makes hidden evaluation admissible. It specifies which actions may be taken, which observations may be used as evidence, and which behavioral relations may be checked. We represent a public behavioral contract as

$$C = (\mathcal{A}_C, \mathcal{O}_C, \mathcal{R}_C). \quad (1)$$

The action set $\mathcal{A}_C$ defines the browser operations admitted by the contract, encompassing control actions, input updates, uploads, role changes, navigation, and reloads. The observation set $\mathcal{O}_C$ defines the public evidence channels available to the evaluator, which include application state, browser metadata, service calls, and rendered projections. The relation set $\mathcal{R}_C$ defines behavioral checks over observations, capturing state transitions, validation behavior, rendered projections, persistence, and authorization effects.

A private scenario τ is admissible, written $\tau \preceq C$, only if all of its references can be resolved through the public contract. Each action must instantiate an element of $\mathcal{A}_C$, each observation reference must be drawn from $\mathcal{O}_C$, and each check must instantiate a relation in $\mathcal{R}_C$. The arguments to these actions and checks may use only declared selectors, roles, routes, service fixtures, upload schemas, tables, state paths, and evidence channels. A private scenario may choose concrete values, order

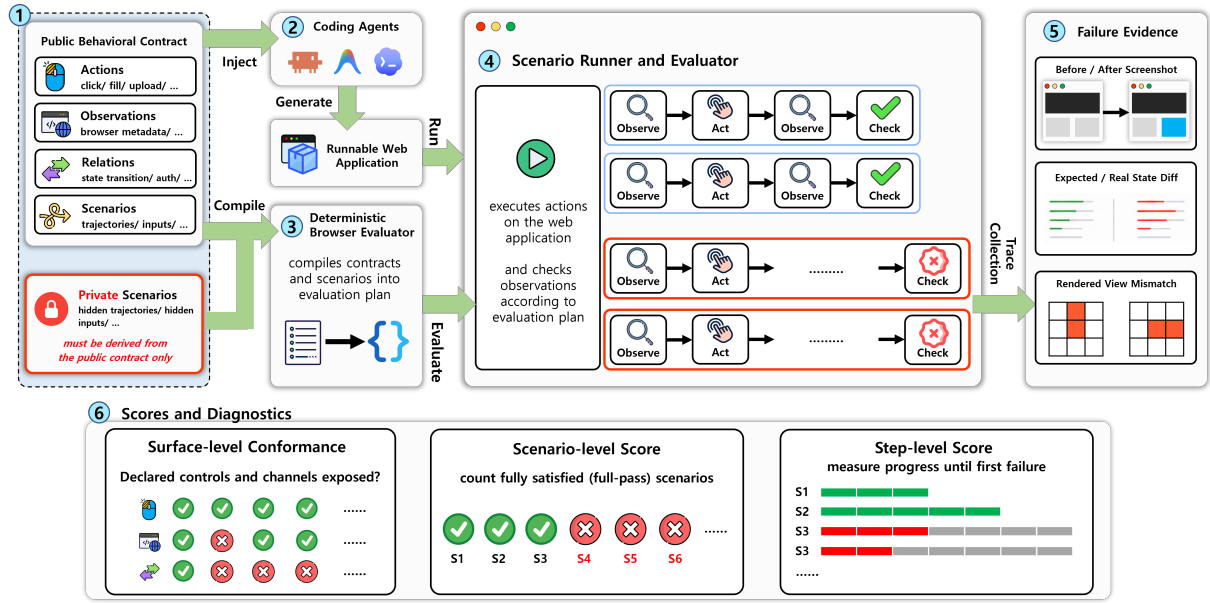


Figure 1: Overview of the ConformWeb. The evaluation operates in six integrated stages: (1) Contract Definition, specifying public behaviors that bound the generation of undisclosed private scenarios; (2) Application Generation, where agents ingest the public contract to build a runnable web application; (3) Evaluation Planning, compiling contracts and hidden workflows into deterministic execution plans; (4) Scenario Execution, running multi-stage browser trajectories while checking live observations; (5) Trace Collection, capturing state diffs and layout mismatches upon any contract violation; and (6) Granular Diagnostics, stratifying scores across surface-level exposure, scenario-level completion, and step-level progress.

declared actions, and compose declared behaviors, but it cannot introduce new product requirements. A scenario that cannot be derived from the contract represents a construction error in the benchmark instance rather than an implementation failure of a generated application.

3.2 Scenario Execution Semantics

For a generated application G and an admissible private scenario τ , we write the sequence as

$$\tau = ((a_i, Q_i))_{i=1}^n, \quad a_i \in \mathcal{A}_C. \quad (2)$$

In this formulation, a_i is a declared browser action and Q_i is the finite set of checks evaluated after that action. Each check $q \in Q_i$ instantiates a behavioral relation from $\mathcal{R}_C$ using paths, values, roles, routes, or evidence channels declared by the contract. Executing τ on G produces a browser trace over the observation surface $\mathcal{O}_C$ expressed as

$$o_0, a_1, o_1, \dots, a_n, o_n. \quad (3)$$

The initial observation o_0 is recorded before the first action, and each subsequent o_i is recorded immediately following action a_i . The application satisfies τ under contract C , written $G \models_C \tau$, when

every individual step passes. For $i = 1, \dots, n$, we define the step-level passage as

$$\text{pass}_i(G, \tau) = \text{exec}_G(a_i) \wedge \text{eval}(Q_i, o_{i-1}, o_i), \quad (4)$$

$$\text{where } \text{eval}(Q_i, o_{i-1}, o_i) = \bigwedge_{q \in Q_i} q(o_{i-1}, o_i).$$

Passing eval requires the joint satisfaction of all behavioral relations checked at step i , taking both o_{i-1} and o_i to evaluate state transitions and persistence effects across the action boundary. Scenario satisfaction is consequently established as

$$G \models_C \tau \iff \forall i \in 1, \dots, n, \text{pass}_i(G, \tau). \quad (6)$$

The term $\text{exec}_G(a_i)$ explicitly integrates actionable execution into the conformance metric. Rather than relying on evaluator heuristics, incorporating $\text{exec}_G(a_i)$ ensures that interactions are actively permitted at the current workflow state and that observations are captured at the exact junctions declared by the contract.

3.3 Scores and Failure Evidence

The main benchmark scores are computed over private scenarios, while surface-level exposure and public scenario conformance serve as diagnostic

levels to isolate missing interaction surfaces and disclosed workflow failures from held-out composition breakdowns. Let $T_C = \{\tau_1, \dots, \tau_m\}$ be the private scenario set used to score implementations for contract C . ConformWeb aggregates private evaluation through two primary metrics. The scenario-level completion score counts fully satisfied private scenarios, and the step-level progress score measures execution advancement prior to the first failure:

$$s_{\text{scen}}(G) = \frac{\sum_{\tau \in T_C} w_{\tau} \mathbf{1}[G \models_C \tau]}{\sum_{\tau \in T_C} w_{\tau}}, \quad (7a)$$

$$s_{\text{step}}(G) = \frac{\sum_{\tau \in T_C} w_{\tau} p_{\tau}(G)}{\sum_{\tau \in T_C} w_{\tau}}. \quad (7b)$$

In our framework, w_{τ} represents the scenario weight, set uniformly across all instances in our experiments. For a scenario $\tau = ((a_i, Q_i))_{i=1}^{n_{\tau}}$, let $\ell_{\tau}(G)$ be the length of the longest prefix where every step passes, defining the progress ratio as

$$p_{\tau}(G) = \frac{\ell_{\tau}(G)}{n_{\tau}}. \quad (8)$$

Consequently, $p_{\tau}(G) = 1$ corresponds exactly to full satisfaction ($G \models_C \tau$), whereas lower values directly quantify how far the application progresses before encountering its first failed action or check.

4 Benchmark Suite

4.1 Application Families and Tiers

ConformWeb instantiates the protocol across 24 distinct benchmark instances formed by crossing six application families with four complexity tiers, scaling to 756 scenarios over 6,433 semantic steps. Rather than evaluating isolated tasks, each instance demands that coding agents implement a fully functional application solely from a public specification, which is then challenged under undisclosed private workflows. This design ensures that the evaluation measures genuine conformance to declared behavioral contracts rather than the superficial imitation of a hidden reference application.

As summarized in Table 1, the suite isolates specific failure modes by progressively increasing behavioral pressure across the four tiers. Tier A establishes local consistency, Tiers B and C introduce operational views and policy constraints, and Tier D pushes execution to final release stages where long-horizon state synchronization must remain absolute. This structured depth creates an exceptionally high bar for current LLMs, evidenced by

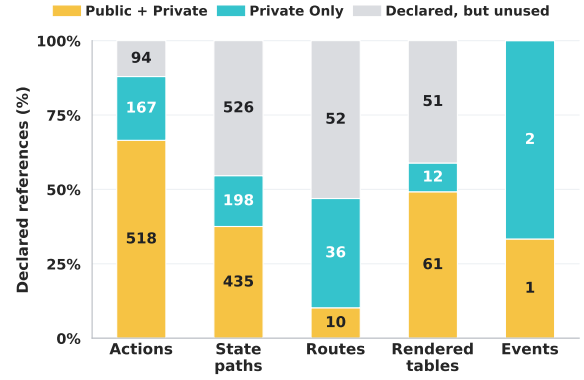


Figure 2: Distribution of declared contract references across evaluation scenarios. Yellow segments indicate references shared between public and private scenarios, cyan indicates private-only references, and gray indicates declared references left entirely unscored.

a baseline full-pass rate of merely 11.7%. As detailed in Section 5, even state-of-the-art frontier models consistently fracture under this pressure, revealing clear performance gaps when multi-view consistency is rigorously demanded.

4.2 Contract Reference Coverage

To guarantee that ConformWeb evaluates genuine programmatic generalization rather than memory memorization, private scenarios must be hidden in their sequential composition but fully exposed in their behavioral scope. To audit this boundary, we track how scenario references—encompassing actions, state paths, routes, rendered tables, and event channels—resolve back to the public contract. Figure 2 details this structural distribution across the entire suite, confirming that every private test segment remains strictly grounded within the pre-declared public contract.

The empirical distribution reveals a deliberate generalization pressure. As shown by the yellow blocks, a significant portion of private checking intensely reuses references already disclosed in public workflows, forcing models to compose known primitives into new stateful orders. Crucially, the cyan segments represent held-out behavioral capabilities; these are entirely unseen workflows that models must still fulfill because their underlying routes, state paths, and events were explicitly declared in the initial contract. Finally, the gray segments verify that the contract is not a thin wrapper or a disguised cheat-sheet for the scoring engine; it leaves ample declared surface unused, penalizing agents that rely on surface-level visual imitation rather than robust contract fulfillment.

Table 1: Complexity tiers in ConformWeb. The tiers progress from short local workflows to longer sequences involving operational views, policy constraints, deterministic services, uploads, persistence, and release synchronization.

Tier	Name	Target scope	Behavioral pressure	Scoring evidence
A	Consumer	Short user workflows	Local state, validation, simple service responses, route transitions, reload behavior	State deltas, validation outcomes, totals, persistence checks
B	Operational	Staff and fulfillment workflows	Operational views, ordered collections, derived counters, status changes, rendered tables	Collection updates, workflow status, counters, rendered rows
C	Regulated	Role and policy workflows	Permission gates, uploads, deterministic services, conflict branches, persistent records	Unauthorized effects, parsed upload state, service outcomes, policy traces
D	Release	Final approval and release workflows	Public ledgers, browser history, re-open behavior, release synchronization, long action chains	Multi-view consistency, history and reload behavior, final ledger checks

4.3 Instance Construction and Validation

Each instance is admitted only after rigorous executable validation. The product premise and public contract are authored first, and both public and private scenarios are subsequently compiled against these declared contract elements. Any scenario referencing undeclared behavior is rejected during construction. For accepted instances, a reference implementation is developed solely from the public materials without access to private scenario files. This reference application serves as an executable witness, confirming that the public contract provides sufficient specification to realize the private scenarios.

While the reference application proves theoretical implementability, actual blind generation outcomes provide empirical evidence that the private scenarios are entirely fair and attainable without leaking evaluation artifacts. As documented in Panel A of Table 2, 252 out of 2,160 total generated applications successfully achieved a 100% full-pass rate across all scoring scenarios. This collective metric serves as an exceptionally rigorous baseline because it demands concurrent satisfaction of both disclosed public and private scenarios within a single compiled runtime. These full-pass runs are not isolated anomalies; they span every complexity tier, distributing as 116/540 in Tier A, 82/540 in Tier B, 44/540 in Tier C, and 10/540 in Tier D. These empirical distributions demonstrate that while private scenarios remain perfectly implementable from public contracts, current frontier models satisfy them with decreasing frequency as sequential workflow pressure intensifies. Detailed target statistics, representative surfaces, and validation results are reported in Appendices B–H.

5 Evaluation

5.1 Experimental Setup

Models and baseline selection. We evaluate nine representative LLMs spanning three leading commercial providers directly over the benchmark suite detailed in Section 4. We select the latest model families from OpenAI (?), Anthropic (Anthropic, 2026), and Google (?), categorizing them into three capacity bands based on operational cost and computing scale. The Mini band contains GPT-5.4-nano, Claude Haiku 4.5, and Gemini 3.1 Flash Lite. The Turbo band evaluates GPT-5.4-mini, Claude Sonnet 4.6, and Gemini 3.1 Flash. The Frontier band includes the flagship reasoning capabilities of GPT-5.4, Claude Opus 4.6, and Gemini 3.1 Pro Preview. To mitigate inference variance and ensure statistical robustness, each model generates ten independent samples per instance, producing a unified matrix of exactly 2,160 execution runs.

Generation and evaluation pipeline. For every family-tier instance, each model receives identical public inputs consisting of the product premise, contract, and disclosed scenarios, while the fixed prompt template strictly excludes private scenarios, reference implementations, traces, and feedback to ensure single-shot code synthesis without multi-turn repair. All generated applications are collected as standard static web artifacts (HTML, CSS, and JS). The evaluation pipeline employs a deterministic Playwright and Chromium runner that initializes each scenario in a fresh browser context to execute the compiled browser plan, record contract-declared state transitions and visual evidence, and apply rule-based assertions.

Table 2: Quantitative evaluation of LLM behavioral conformance and complexity tier stress. Panel A establishes the baseline distribution across the complete execution scope. Panel B itemizes detailed conformance metrics and full-pass rates across capability bands and complexity tiers. All reported values reflect explicit raw execution counts without statistical inference or group-level interpolation.

Group	Model	Full pass	S_{scen}	S_{step}	Complexity Tier			
					Tier A	Tier B	Tier C	Tier D
Panel A: complete blind full-pass evidence								
All	Complete aggregate	252/2160 (11.7%)	36.7%	49.4%	116/540 (21.5%)	82/540 (15.2%)	44/540 (8.1%)	10/540 (1.9%)
Panel B: model-level conformance and tier stress								
Mini	Group aggregate	6/720 (0.8%)	10.9%	24.8%	1/180 (0.6%)	2/180 (1.1%)	3/180 (1.7%)	0/180 (0.0%)
	GPT-5.4 nano	5/240 (2.1%)	17.5%	32.0%	0/60 (0.0%)	2/60 (3.3%)	3/60 (5.0%)	0/60 (0.0%)
	Claude Haiku 4.5	1/240 (0.4%)	14.3%	31.5%	1/60 (1.7%)	0/60 (0.0%)	0/60 (0.0%)	0/60 (0.0%)
	Gemini 3.1 Flash Lite	0/240 (0.0%)	0.9%	10.9%	0/60 (0.0%)	0/60 (0.0%)	0/60 (0.0%)	0/60 (0.0%)
Turbo	Group aggregate	72/720 (10.0%)	31.9%	43.9%	35/180 (19.4%)	22/180 (12.2%)	12/180 (6.7%)	3/180 (1.7%)
	GPT-5.4 mini	3/240 (1.2%)	4.3%	13.5%	3/60 (5.0%)	0/60 (0.0%)	0/60 (0.0%)	0/60 (0.0%)
	Claude Sonnet 4.6	46/240 (19.2%)	55.0%	67.4%	19/60 (31.7%)	14/60 (23.3%)	10/60 (16.7%)	3/60 (5.0%)
	Gemini 3.1 Flash	23/240 (9.6%)	36.5%	50.9%	13/60 (21.7%)	8/60 (13.3%)	2/60 (3.3%)	0/60 (0.0%)
Frontier	Group aggregate	174/720 (24.2%)	67.2%	79.3%	80/180 (44.4%)	58/180 (32.2%)	29/180 (16.1%)	7/180 (3.9%)
	GPT-5.4	47/240 (19.6%)	68.4%	81.0%	23/60 (38.3%)	17/60 (28.3%)	3/60 (5.0%)	4/60 (6.7%)
	Claude Opus 4.6	69/240 (28.8%)	70.8%	82.3%	32/60 (53.3%)	22/60 (36.7%)	15/60 (25.0%)	0/60 (0.0%)
	Gemini 3.1 Pro Preview	58/240 (24.2%)	62.3%	74.7%	25/60 (41.7%)	19/60 (31.7%)	11/60 (18.3%)	3/60 (5.0%)

Metrics. We report the full pass rate along with strict scenario-level conformance S_{scen} and step-level progress S_{step} . The full pass metric requires every scoring scenario within an instance to succeed. The S_{scen} score credits only fully passed scoring scenarios while S_{step} measures the completed execution prefix before the initial behavioral failure. To analyze performance across varying complexity, we further disaggregate these metrics by complexity tiers A through D, reflecting increasing behavioral requirements. All aggregates are computed over the complete 2,160-run experimental scope to ensure consistency with the results presented in Table 2. Probe scenarios are excluded from formal denominators. Detailed reproducibility metadata and run accounting information are provided in Appendix K.

5.2 Main Results

Full conformance is rare. Across the 2,160 execution runs, only 252 pass the full scoring set, yielding an 11.7% full pass rate. Partial metrics are substantially higher as scenario level conformance reaches 36.7% and step level progress reaches 49.4%. This gap is our central empirical signal. Generated applications often instantiate the requested surfaces and complete meaningful workflow prefixes but still fail to preserve all declared behaviors end to end. Detailed breakdowns across

all application families are provided in Appendix D. Full pass is strict by design because a user facing workflow fails if a single state transition, permission check, or final ledger update contradicts a prior obligation.

Capability improves progress but does not guarantee conformance. Table 2 demonstrates that stronger models advance further through workflows yet improvement is more pronounced in partial progress than in complete conformance. The Mini band achieves a 0.8% full pass rate with 6 per 720 runs. The Turbo band reaches 10.0% with 72 per 720 runs. The Frontier band attains 24.2% with 174 per 720 runs. This scaling trend is mirrored in partial metrics where Frontier models reach 67.2% scenario conformance and 79.3% step progress. This result rules out a simple surface generation interpretation. Even the most capable models often preserve local obligations but fail when later operations depend on earlier state, roles, or service persistence.

Higher complexity tiers stress workflow consistency. The tiered results follow the design in Section 4. Tier A has 116 full-Results follow the complexity progression defined in Section 4. Full pass rates decline across tiers as Tier A has 21.5% with 116 per 540 runs, Tier B has 15.2% with 82 per 540 runs, Tier C has 8.1% with 44 per 540 runs, and

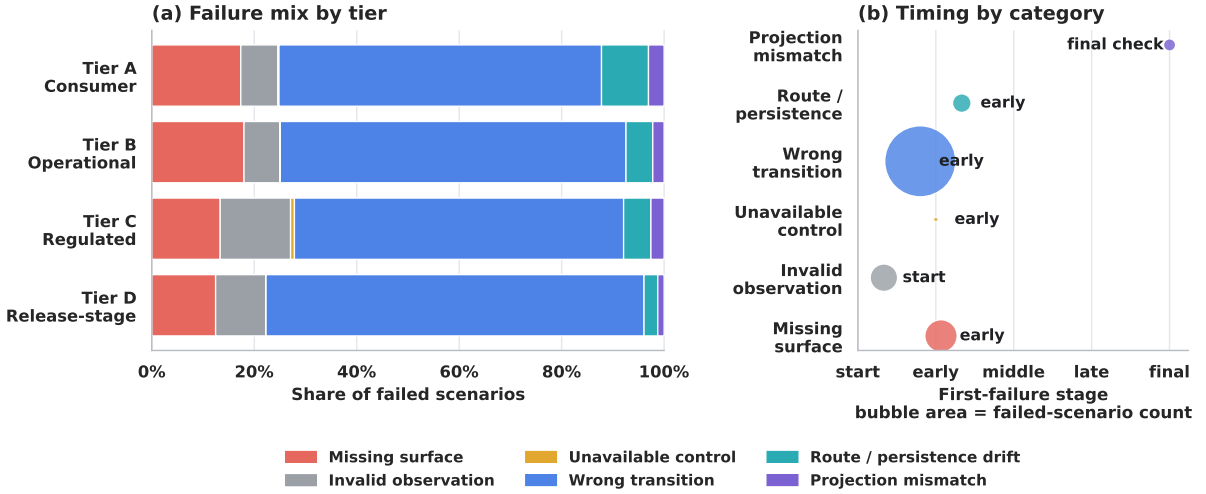


Figure 3: Taxonomy and temporal dynamics of first behavioral failures. (a) Relative distribution of the six failure categories across complexity tier A through D. (b) Temporal positioning of each failure category within normalized workflows, where the bubble area corresponds to the failed scenario count.

Tier D has 1.9% with 10 per 540 runs. This decline is expected by construction as later tiers require state, permissions, and browser views to remain synchronized across longer interactions. Table 2 shows this pressure persists across all capability bands. Higher tiers are not merely longer tasks. They introduce tight coupling among browser state, role policies, and service responses, effectively separating shallow surface completion from sustained behavioral conformance.

5.3 Failure Analysis

ConformWeb failures differ in both type and timing. We define progress as the length of the successful prefix sequence that a model completes before the first behavioral violation. For each failed scenario, we analyze the first step after that prefix to avoid counting cascading failures triggered by the initial violation and to obtain a comparable diagnosis point. Figure 3 maps these initial breakdown points across application tiers and workflow stages, categorizing them into six contract surfaces ranging from missing interaction elements to late-stage projection mismatches. Within this framework, long horizon drift characterizes cases where one of these fundamental violations surfaces only after a substantial prefix sequence.

The failure patterns validate the benchmark design by demonstrating that errors manifest during extended operational sequences rather than initialization. Failed executions yield operational interfaces capable of initial progress but fail to maintain behavioral coherence across complex interactions.

The main failure mode is a decoupling between interaction and evidence, such as when actions execute without the corresponding state change, state changes occur without updating the rendered view, role modifications lack permission effects, or reloads fail to preserve persistence. Because the user interface retains a plausible surface after a contract violation, these discrepancies evade detection via static code review or screenshot inspection. Consequently, identifying these flaws requires dynamic browser execution and contract grounded assertions. More detailed empirical breakdowns are provided in Appendices E and I.

6 Conclusion

ConformWeb evaluates agent-generated web applications by utilizing private browser trajectories grounded in public behavioral contracts. Across 2,160 evaluation runs spanning 24 instances, 756 scenarios, and 6,433 semantic steps, only 252 generations achieve full conformance, yielding an 11.7% pass rate. The contrast between high partial progress and the low full-conformance rate highlights a systemic vulnerability where scaling capability fails to guarantee consistency due to a structural decoupling between interaction and evidence. Because these violations evade static inspections by leaving the interface visually intact, ConformWeb establishes a new evaluation paradigm. It shifts the testing unit from isolated agent trajectories to full-system behavioral conformance under hidden workflow compositions.

References

- Anthropic. 2026. The Claude Model Card and Evaluation Suite. Technical report, Anthropic. Accessed: 2026-05-26.
- Rabiul Awal, Mahsa Massoud, Zichao Li, Aarash Feizi, Suyuchen Wang, Christopher Pal, Aishwarya Agrawal, David Vazquez, Siva Reddy, Juan A Rodriguez, and 1 others. 2025. Webmmu: A benchmark for multimodal multilingual website understanding and code generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Tony Beltramelli. 2018. [pix2code: Generating code from a graphical user interface screenshot](#). In *Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems, EICS '18*, New York, NY, USA. Association for Computing Machinery.
- Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault L De Chezelles, Quentin Cappart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. 2024. Workarena+: Towards compositional planning and reasoning-based common knowledge work tasks. *Advances in Neural Information Processing Systems*, 37:5996–6051.
- Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, and 1 others. 2023. Multipl-e: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 49(7):3675–3691.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- De Chezelles, Thibault Le Sellier, Sahar Omid Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F Xu, Siva Reddy, Quentin Cappart, and 1 others. 2024. The browser-gym ecosystem for web agent research. *arXiv preprint arXiv:2412.05467*.
- Koen Claessen and John Hughes. 2000. Quickcheck: a lightweight tool for random testing of haskell programs. In *Proceedings of the fifth ACM SIGPLAN international conference on Functional programming*, pages 268–279.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114.
- Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, and 1 others. 2024. Workarena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718*.
- Dalal Hammoud, Fadi A Zaraket, and Wes Masri. 2017. Guicop: Approach and toolset for specification-based gui testing. *Software Testing, Verification and Reliability*, 27(8):e1642.
- Amine El hattami, Megh Thakkar, Nicolas Chapados, and Christopher Pal. 2025. [Webarena verified: Reliable evaluation for web agents](#). In *Workshop on Scaling Environments for Agents*.
- Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6864–6890.
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and 1 others. 2021. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*.
- Naman Jain, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, pages 58791–58831.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157.
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. 2024. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905.
- Ron Kohavi and 1 others. 1995. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Ijcai*, volume 14, pages 1137–1145. Montreal, Canada.
- Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. Ds-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning*, pages 18319–18345. PMLR.

708	Zhiyu Lin, Zhengda Zhou, Zhiyuan Zhao, Tianrui Wan,	Zhiyuan Peng, Wei Tao, Xin Yin, Chenhao Ying, Yuan	764
709	Yilun Ma, Junyu Gao, and Xuelong Li. 2025. We-	Luo, and Yiwen Guo. 2026. Playcoder: Making	765
710	buibench: a comprehensive benchmark for evaluating	llm-generated gui code playable. <i>arXiv preprint</i>	766
711	multimodal large language models in webui-to-code.	<i>arXiv:2604.19742</i> .	767
712	In <i>Findings of the Association for Computational</i>		
713	<i>Linguistics: ACL 2025</i> , pages 15780–15797.		
714	Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Ling-	Ruchir Puri, David S Kung, Geert Janssen, Wei Zhang,	768
715	ming Zhang. 2023a. Is your code generated by chat-	Giacomo Domeniconi, Vladimir Zolotov, Julian	769
716	gpt really correct? rigorous evaluation of large lan-	Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker,	770
717	guage models for code generation. In <i>Proceedings</i>	and 1 others. 2021. Codenet: A large-scale ai for	771
718	<i>of the 37th International Conference on Neural In-</i>	code dataset for learning a diversity of coding tasks.	772
719	<i>formation Processing Systems, NIPS '23</i> , Red Hook,	<i>arXiv preprint arXiv:2105.12655</i> .	773
720	NY, USA. Curran Associates Inc.		
721	Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang,	Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang,	774
722	Ruochen Xu, and Chenguang Zhu. 2023b. G-eval:	Ruibao Liu, and Diyi Yang. 2025. Design2code:	775
723	Nlg evaluation using gpt-4 with better human align-	Benchmarking multimodal code generation for au-	776
724	ment. In <i>Proceedings of the 2023 conference on</i>	tomated front-end engineering. In <i>Proceedings of</i>	777
725	<i>empirical methods in natural language processing</i> ,	<i>of the 2025 Conference of the Nations of the Americas</i>	778
726	pages 2511–2522.	<i>Chapter of the Association for Computational Lin-</i>	779
727	Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey	<i>guistics: Human Language Technologies (Volume 1:</i>	780
728	Svyatkovskiy, Ambrosio Blanco, Colin Clement,	<i>Long Papers)</i> , pages 3956–3974.	781
729	Dawn Drain, Daxin Jiang, Duyu Tang, and 1 others.		
730	2021. Codexglue: A machine learning benchmark	Gerrit Jan Tretmans. 1996. Test generation with in-	782
731	dataset for code understanding and generation. <i>arXiv</i>	puts, outputs and repetitive quiescence. <i>Software:</i>	783
732	<i>preprint arXiv:2102.04664</i> .	<i>concepts & tools</i> , 17(3):103–120.	784
733	Xing Han Lù, Zdeněk Kasner, and Siva Reddy. 2024.	Jan Tretmans. 2008. Model based testing with labelled	785
734	Weblinx: Real-world website navigation with multi-	transition systems. In <i>Formal methods and testing:</i>	786
735	turn dialogue. <i>arXiv preprint arXiv:2402.05930</i> .	<i>An outcome of the FORTEST network, revised se-</i>	787
		<i>lected papers</i> , pages 1–38. Springer.	788
736	Xing Han Lù, Amirhossein Kazemnejad, Nicholas	Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin	789
737	Meade, Arkil Patel, Dongchan Shin, Alejandra Zam-	Manku, Vinty Dong, Edward Li, Shashank Gupta,	790
738	brano, Karolina Stańczak, Peter Shaw, Christopher J	Ashish Sabharwal, and Niranjan Balasubramanian.	791
739	Pal, and Siva Reddy. 2025. Agentrewardbench: Eval-	2024. AppWorld: A controllable world of apps and	792
740	uating automatic evaluations of web agent trajec-	people for benchmarking interactive coding agents .	793
741	ories. <i>arXiv preprint arXiv:2504.08942</i> .	In <i>Proceedings of the 62nd Annual Meeting of the</i>	794
		<i>Association for Computational Linguistics (Volume 1:</i>	795
742	Zimu Lu, Yunqiao Yang, Houxing Ren, Haotian Hou,	<i>Long Papers)</i> , pages 16022–16076, Bangkok, Thai-	796
743	Han Xiao, Ke Wang, Weikang Shi, Aojun Zhou,	land. Association for Computational Linguistics.	797
744	Mingjie Zhan, and Hongsheng Li. 2026. Webgen-	Jingyu Xiao, Yuxuan Wan, Yintong Huo, Zixin Wang,	798
745	bench: Evaluating llms on generating interactive and	Xinyi Xu, Wenxuan Wang, Zhiyao Xu, Yuhang Wang,	799
746	functional websites from scratch. <i>Advances in Neu-</i>	and Michael R Lyu. 2025. Interaction2code: Bench-	800
747	<i>ral Information Processing Systems</i> , 38.	marking mllm-based interactive webpage code ge-	801
		neration from interactive prototyping. In <i>2025 40th</i>	802
748	Atif M Memon. 2007. An event-flow model of gui-	<i>IEEE/ACM International Conference on Automated</i>	803
749	based applications for testing. <i>Software testing, veri-</i>	<i>Software Engineering (ASE)</i> , pages 241–253. IEEE.	804
750	<i>fication and reliability</i> , 17(3):137–157.	Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan	805
751	Atif M Memon, Martha E Pollack, and Mary Lou Soffa.	Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun	806
752	2002. Hierarchical gui test case generation using	Cheng, Dongchan Shin, Fangyu Lei, and 1 others.	807
753	automated planning. <i>IEEE transactions on software</i>	2024. Osworld: Benchmarking multimodal agents	808
754	<i>engineering</i> , 27(2):144–155.	for open-ended tasks in real computer environments.	809
		<i>Advances in Neural Information Processing Systems</i> ,	810
755	Ali Mesbah, Arie Van Deursen, and Stefan Lenselink.	37:52040–52094.	811
756	2012. Crawling ajax-based web applications through	Kai Xu, YiWei Mao, XinYi Guan, and ZiLong Feng.	812
757	dynamic analysis of user interface state changes.	2025. Web-bench: A llm code benchmark based	813
758	<i>ACM Transactions on the Web (TWEB)</i> , 6(1):1–30.	on web standards and frameworks. <i>arXiv preprint</i>	814
		<i>arXiv:2505.07473</i> .	815
759	Augustus Odena, Charles Sutton, David Martin Do-	Shunyu Yao, Howard Chen, John Yang, and Karthik	816
760	han, Ellen Jiang, Henryk Michalewski, Jacob Austin,	Narasimhan. 2022. Webshop: Towards scalable real-	817
761	Maarten Paul Bosma, Maxwell Nye, Michael Terry,	world web interaction with grounded language agents.	818
762	and Quoc V. Le. 2021. Program synthesis with large	<i>Advances in Neural Information Processing Systems</i> ,	819
763	language models. In <i>n/a</i> , page n/a, n/a. N/a.	35:20744–20757.	820

- Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Shulin Xin, Linhao Zhang, Qi Liu, Li Aoyan, Lu Chen, Xiaojian Zhong, and 1 others. 2026. Multi-swe-bench: A multilingual benchmark for issue resolving. *Advances in Neural Information Processing Systems*, 38.
- Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. Gpt-4v (ision) is a generalist web agent, if grounded. *arXiv preprint arXiv:2401.01614*.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, and 1 others. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, and 1 others. 2024. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, volume 2024, pages 15585–15606.
- Hongda Zhu, Yiwen Zhang, Bing Zhao, Jingzhe Ding, Siyao Liu, Tong Liu, Dandan Wang, Yanan Liu, and Zhaojian Li. 2025. Frontendbench: A benchmark for evaluating llms on front-end development via automatic evaluation. *arXiv preprint arXiv:2506.13832*.
- Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widayarsi, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, and 1 others. 2025. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In *International Conference on Learning Representations*, volume 2025, pages 66602–66656.

A Contract Language and Browser Evaluation

ConformWeb uses a structured contract language so that application generation and evaluation share the same public specification. The contract is available during application generation, but private scenario instances, action orderings, concrete values, reference implementations, screenshots, traces, and failure feedback are not disclosed.

Artifact level auditability requires connecting the formal view in Section 3 to the packaged benchmark files. The main paper abstracts a contract as actions, observations, and behavioral relations. The artifact represents that abstraction in YAML, checks private scenario admissibility against the public contract, and lowers scenario steps into browser plans. This makes the disclosure boundary explicit. Private scenarios may select values and compose declared behaviors, but the compiler rejects references to undeclared product requirements.

A.1 Contract Files and Disclosure Boundary

The packaged benchmark instances follow the same file convention. Each instance contains a public contract.dsl.yaml file and scenario files when available. Listing 1 shows the top level contract shape. The public contract declares the admissible interaction and evidence surfaces. Private scenarios may choose values and compose declared behaviors, but they may not introduce new selectors, routes, state paths, service responses, table schemas, or persistence obligations.

The public and private file split lets ConformWeb evaluate conformance without changing the task after generation. The application generation process receives the vocabulary of the application, including components, routes, roles, state paths, deterministic services, rendered tables, and the product premise. The evaluator then tests compositions of that vocabulary. A failure is therefore interpretable as a conformance failure to the public contract rather than as a penalty for not guessing an undisclosed feature.

Listing 1: Public contract file skeleton.

```
dsl_version: "3.0.0"
app:
  id: <target id>
  name: <display name>
  capability: <short capability label>
  ui_ux_brief: <product premise or interface brief>
runtime:
  target: web
  state_probe:
```

```
expression: <JavaScript expression returning observable
  application state>
pages: {...}
session: {...}
persistence: {...}
actors: {...}
services: {...}
events: {...}
tables: {...}
state:
  initial: {...}
  schema: {...}
components:
  <component_id>: {...}
```

A.2 YAML Contract Excerpts

The YAML excerpts below are representative fragments of the public contracts and private scenarios used to construct the benchmark. They are not synthetic examples. Listing 2 shows how the product premise, routes, and roles appear in a public contract. Listing 3 shows how a role gated action declares both allowed and unauthorized effects. Listing 4 shows how a private scenario composes declared actions and checks.

The examples highlight three design choices. The product premise is stored as a public interface brief field, not as a visual oracle. Role gated behavior is expressed through explicit allowed and unauthorized effects. A private scenario keeps the target behavior visible while hiding the private scoring trajectory.

Listing 2: Product premise, routes, and roles in a public contract.

```
app:
  id: campus_registrar_command_tier_a
  name: Campus Registrar Command Tier A
  capability: course_registration_planning
  ui_ux_brief: |
    Campus Registrar Command Tier A should look like a real
    university registration and advising product, not a
    generic admin dashboard. A desktop screenshot should
    immediately show a course catalog, weekly schedule
    grid, credit load meter, degree audit checklist,
    waitlist lane, advisor override panel, and student
    enrollment ledger.
runtime:
  pages:
    catalog: {path: /#/catalog}
    planner: {path: /#/planner}
    audit: {path: /#/audit}
  session:
    role_path: active_role
    roles: [student, advisor]
```

Listing 3: Role gated action with allowed and unauthorized effects.

```
components:
  advisor_override_button:
    selector: '[data-testid="advisor-override"]'
    page: audit
    actions:
      click:
        allowed_roles: [advisor]
        preconditions:
          - {op: one_of, path: eligibility_status, values:
            [advisor_required]}
```



```

974     effects:
975       - {op: equals, path: eligibility_status, value:
976         eligible}
977       - {op: equals, path: approval_status, value:
978         advisor_approved}
979       unauthorized_effects:
980       - {op: changed_by, path: denied_count, by: 1}
981       - {op: equals, path: last_action, value:
982         unauthorized_advisor}

```

Listing 4: Private scenario composed from declared actions and checks.

```

984 - id: private_01_ac_quote_and_acceptance_journey
985   tier: journey
986   steps:
987     - {do: reset_button.click}
988     - {do: select_ac_button.click}
989     - {do: go_booking_button.click}
990     - {do: slot_today_button.click}
991     - do: customer_name_input.fill
992       input: {value: Mina Park}
993     - do: address_input.fill
994       input: {value: 88 Cedar House}
995     - do: get_quote_button.click
996       expect:
997         - {op: equals, path: quote_amount, value: 18900}
998     - {do: approve_quote_button.click}
999     - {do: book_visit_button.click}
1000     - {do: go_visits_button.click}
1001     - {do: role_pro_button.click}
1002     - do: pro_accept_button.click
1003       expect:
1004         - {op: equals, path: booking_status, value:
1005           accepted}
1006

```

A.3 Scenario Flow Examples

The following table shows how a scenario becomes a browser evaluable plan. The first row is a compact public upload scenario from the Campus family. The next rows show longer private journeys only at the phase level, illustrating how later tiers compose many declared behaviors without exposing hidden values in the main paper.

A.4 Scenario Step Lowering

A component action step is not evaluated directly from the YAML file. The contract compiler first inserts a pre action snapshot and then emits the browser action with post action checks. Listing 5 shows a schematic lowering of the HomeFix quote step from Listing 4.

The lowering supports both step progress and first failure evidence. The snapshot fixes the before state used by delta checks, while the action step records the after state, rendered evidence, and screenshot used by the assertion engine. The same mechanism supports late failure diagnosis. A scenario can pass many action and check pairs before a later check fails, and the evaluator still records the first failed step.

Listing 5: Compiled browser plan for one scenario step.

```

1032 - action: snapshot
1033   save_as: dsl_before_7
1034

```

```

- action: click
  component: get_quote_button
  selector: '[data-testid="get-quote"]'
  expect:
    - type: state_equals
      path: quote_amount
      expected: 18900
    - type: state_delta
      path: quote_call_count
      from: dsl_before_7
      by: 1
  contract_refs:
    - contract:get_quote_button.click:effect

```

The lowering supports both step progress and first failure evidence. The snapshot fixes the before state used by delta checks, while the action step records the after state, rendered evidence, and screenshot used by the assertion engine. The same mechanism supports late failure diagnosis. A scenario can pass many action and check pairs before a later check fails, and the evaluator still records the first failed step.

A.5 Compiler and Evaluator Excerpts

The contract compiler first validates the contract and scenario bundle, then builds a web contract and scenario set. Listing 6 shows the compilation entry point, and Listing 7 shows the lowering of one compiled step into browser plan steps.

The browser evaluator executes each compiled plan in a fresh browser context, records before and after observations and screenshots, installs deterministic service routes, and evaluates rule based assertions. Listings 8 and 9 show the capture and state observation structure. Listing 10 shows the admissibility boundary, Listing 11 shows fresh context execution, and Listing 12 shows rendered table capture. These excerpts identify the trust boundary. The browser runner interacts through declared selectors, records declared evidence, and applies deterministic assertions without inspecting source code or asking a language model to judge the trace.

Listing 6: Contract compilation entry point.

```

def compile_dsl_to_web_suite(contract_dsl, scenarios_dsl):
    compiled = compile_bundle(contract_dsl, scenarios_dsl)
    return build_web_contract(contract_dsl,
        build_web_scenarios(scenarios_dsl, compiled)

```

Listing 7: Lowering one scenario step into browser plan steps.

```

snapshot_name = f"dsl_before_{step.index}"
snapshot_step = {
    "action": "snapshot",
    "save_as": snapshot_name,
    "expect": [_precondition_to_assertion(c) for c in
        step.preconditions],
}

```

Family / tier	Scenario kind	Flow summary
Campus / B	public upload check	reset → go to petitions → upload transfer CSV → check transfer table rows and credit projections
HomeFix / D	private release journey	reset → select repair issue → book slot → request quote → homeowner approval → dispatcher role switch → assign technician → parts readiness → QA/insurance/warranty steps → reload or reseal checks
Clinic / D	private incident command	filter critical patient → assign isolation → escalation and risk scoring → insurance/auth roles → clinician and operations signoffs → final incident-command seal checks

Table 3: Representative scenario flows. The evaluator sees concrete browser actions and assertions; the main paper reports phase-level summaries to keep hidden scoring instances separate from the narrative.

```

action_step = {
  "action": _web_action(step.action),
  "dsl_mode": step.mode,
  "expect": [_effect_to_assertion(c, snapshot_name) for c
    in step.checks],
  "contract_refs": [c.source for c in step.checks],
}

```

The browser evaluator executes each compiled plan in a fresh browser context, records before and after observations and screenshots, installs deterministic service routes, and evaluates rule based assertions. The excerpts below are not meant to document the full implementation. They identify the trust boundary. The browser runner interacts through declared selectors, records declared evidence, and applies deterministic assertions without inspecting source code or asking a language model to judge the trace.

Listing 8: Evaluator loop for one scenario step.

```

before_state = self._capture_state(page)
before_screenshot = self._screenshot(page, scenario_id,
  index, "before")
if action != "snapshot":
  self._perform_action(page, action, step)
after_state = self._capture_state(page)
after_screenshot = self._screenshot(page, scenario_id,
  index, "after")
assertions = evaluate_assertions(
  list(step.get("expect", [])),
  after_state=after_state,
  snapshots=self.snapshots,
)

```

Listing 9: Observation capture with browser, service, and table evidence.

```

state = page.evaluate(self.contract.state_expression)
state["__browser"] = self._capture_browser_state(page)
state["__services"] = {"calls": list(self.service_calls)}
state["__tables"] = self._capture_table_state(page)

```

Listing 10: Admissibility check for undeclared references.

```

compiled = compile_bundle(contract_dsl, scenarios_dsl)
# compile_bundle parses the public contract first: state
  schema, pages,
# actors, services, events, tables, session, persistence,
  and components.
# Each scenario step is then accepted only if its
  references are declared.

```

Listing 11: Fresh browser context for each scenario.

```

for scenario in self.scenarios.scenarios:
  context = browser.new_context(viewport={"width": 1280,
    "height": 900})
  self.service_calls = []
  self._install_service_routes(context)
  pages = self._open_scenario_pages(context, app_url,
    scenario)
  result = self._run_scenario(pages, scenario, logger)
  scenario_results.append(result)
  context.close()

```

Listing 12: Rendered table capture through declared selectors.

```

const root = document.querySelector(table.selector);
const rows =
  Array.from(root.querySelectorAll(table.row_selector));
for (const row of rows) {
  const rowId = row.getAttribute(rowIdAttribute);
  captured.order.push(rowId);
  captured.rows[rowId] = cells;
}

```

A.6 Contract Field Groups

Table 4 summarizes major contract fields used across benchmark instances. The protocol is implementation agnostic. An application may use any internal state representation or architecture, provided that it exposes the declared interaction and observation surfaces. In the experiments, submitted applications use the common static app format described in Section 5. The table should be read as a checklist of evidence channels rather than as a programming model. For example, an application may store cart items in any internal format, but it must expose the declared observable state path and rendered projection when those fields are part of the contract.

A.7 Behavioral Relation Types

Private scenarios are compiled into browser actions and rule based assertions. Table 5 lists representative relation types. The relation vocabulary is designed to check behavioral conformance rather than source code organization or visual polish. These relations also explain why screenshots alone are

Field group	Purpose	Examples of obligations
Components and selectors	Declare the interaction surface available to the evaluator.	Buttons, forms, filters, upload controls, role switches, confirmation controls.
Actions	Define admissible browser operations.	Click, fill, select, upload file, change role, navigate, reload, traverse history.
Observable state paths	Expose auditable application state.	Cart totals, booking status, row counts, validation maps, approval status, ledger entries.
Routes and browser meta-data	Specify navigation and route obligations.	Current page, expected route after action, back/forward behavior, reload persistence.
Roles and permissions	Specify which actors can perform or observe actions.	Manager-only approval, registrar override, technician dispatch, unauthorized no-op or denial effects.
Service fixtures	Make external dependencies deterministic.	Quote lookup, inventory service, eligibility checks, dispatch readiness, campaign budget readiness.
Uploads and parsing	Specify file-derived behavior.	CSV rows, asset lists, manifest updates, replacement semantics, derived counters.
Rendered projections	Link observable state to rendered evidence.	Table rows, ordered lists, cells, badges, totals, operational ledgers.
Persistence and history	Specify behavior across browser events.	Reload state, route preservation, history traversal, persistent approvals or cleared state.

Table 4: Major contract field groups used in ConformWeb. Application generation receives these fields as part of the public behavioral contract. Private scenarios instantiate and compose only declared behaviors.

insufficient. Many failures involve disagreement among observable state, route, service calls, and rendered projections even when the page looks plausible.

A.8 Scenario Compilation Pattern

A private scenario is a sequence of scenario steps. Each step is lowered to an executable browser plan using the pattern $\text{OBSERVE}_{pre} \rightarrow \text{ACT} \rightarrow \text{OBSERVE}_{post} \rightarrow \text{CHECK}$. Optional preconditions and wait conditions are inserted only when declared by the contract or scenario. Table 6 shows a representative compilation pattern. Most of the benchmark difficulty comes not from evaluator heuristics, but from composing ordinary browser actions over longer stateful workflows.

A.9 Scenario Step Forms

The scenario files use a small set of step forms. Component steps invoke `component.action`. Browser lifecycle steps exercise navigation, reload, and history. Event steps dispatch declared external events. Table steps assert rendered table evidence. Listing 13 shows representative forms. All paths, components, pages, roles, tables, services, and events must be declared in the public contract before a scenario can be compiled.

```
# Component action step
- do: add_item_button.click
  input: {value: lettuce}
  mode: normal
  expect:
    - {op: changed_by, path: cart_count, by: 1}

# Unauthorized branch over a declared role-gated action
- do: advisor_override_button.click
  mode: unauthorized
  expect:
    - {op: changed_by, path: denied_count, by: 1}

# Browser lifecycle step
- browser: reload
  expect:
    - {op: unchanged, path: cart_items}

# Table evidence step
- table: order_grid
  expect_order: [ORD-1, ORD-2]
  expect_row_count: 2
```

B Benchmark Suite Statistics

ConformWeb contains six application families and four complexity tiers per family, yielding 24 benchmark instances. The main paper describes the design intent of these families. This appendix provides quantitative characterization of the frozen benchmark bundle, including observable state schemas, routes, actors, services, uploads, rendered tables, and private scenario trajectories whose size and composition vary across tiers and families.

Listing 13: Representative scenario step forms.

Relation type	What it checks	Example failure
State equality / membership	Observable state has an expected value or belongs to an expected set.	A status remains draft after a required approval action.
State delta / append	Observable state changes by a declared amount or appends a declared object.	A cart count changes but the corresponding total does not.
Validation relation	Validation maps and errors match the declared input conditions.	An invalid field is accepted without a declared error.
Route relation	Browser route or page identity matches the expected route.	A workflow panel appears but the route remains stale.
Persistence relation	State and route obligations hold after reload or history traversal.	A reload preserves the screen but loses public workflow state.
Rendered projection relation	Rendered evidence agrees with observable state.	A table row is stale, missing, incorrectly ordered, or inconsistent with state.
Unauthorized no-op / denial	Role-gated actions produce declared no-op or denial effects.	A manager-only action mutates state for an unauthorized actor.
Service fixture relation	Declared deterministic service interactions are reflected in state and UI.	A quote or readiness response is ignored or partially applied.

Table 5: Representative relation types compiled into deterministic browser assertions. These relations are derived from public contract fields and do not introduce hidden product requirements.

Stage	Evaluator operation	Evidence recorded
Precondition	Confirm that declared route, role, state, or UI precondition holds.	Public state snapshot, browser route, rendered evidence, screenshot.
Action	Execute a declared browser action through the declared selector or operation.	Action metadata, actor/role, selector, upload or service fixture metadata.
Wait	Wait for declared asynchronous completion or observation point.	Timing status, service call log, post-action stabilization evidence.
Post-observation	Record the state of the application after the action.	Public state, browser metadata, rendered UI evidence, service interactions.
Check	Evaluate rule-based relations over before/after observations.	Passed checks or first-failure relation, expected value, observed value.

Table 6: Representative compilation of a semantic scenario step into deterministic browser evaluation. The compiler lowers declared actions and relations; it does not add new product requirements.

B.1 Contract Bundle Inventory

The frozen benchmark bundle contains the YAML artifacts used to define the benchmark instances. Table 7 summarizes the contract and scenario inventory by tier. Table 8 lists each benchmark instance with raw contract feature counts. These counts are descriptive. Generated applications are scored only by the compiled browser scenarios.

The tier inventory shows the intended scaling pattern. Median observable state paths rise from 28 in Tier A to 68 in Tier D, median component counts rise from 23 to 41, and median rendered table counts rise from 2.5 to 8.5. Private scenario counts also increase across later tiers. The numbers do not define difficulty by size alone, but they show that later tiers expose more places where state, route, role, and rendered evidence can diverge.

The raw inventory also clarifies why family-level

results should not be read as domain rankings. Clinical and Campus targets, for example, include many role and table obligations in later tiers, while Grocery stresses cart and fulfillment consistency with fewer upload obligations. HomeFix includes long private journeys in Tiers C and D even though its public contract files are smaller than several other families. The benchmark therefore varies both the amount of declared surface and the kind of behavioral coupling being composed.

B.2 Benchmark Complexity Across Families and Tiers

Figure 4 gives a family and tier summary of contract complexity. The summary score is descriptive and is not used to score generated applications. It shows how later tiers accumulate pressures such as longer scenarios, additional routes, role dependent actions, services, uploads, rendered projections,

Tier	Name	Contracts	Public scens.	Private scens.	Med. paths	Med. comps.	Med. pages	Med. roles	Upload acts.	Med. tables
A	Consumer	6	58	84	28	23	3	2	1	2.5
B	Operational	6	64	108	42.5	29	4	4	4	5
C	Regulated	6	85	145	56	35	5	5	10	6.5
D	Release-stage	6	87	176	68	41	5.5	5.5	17	8.5

Table 7: YAML artifact inventory by tier in the frozen benchmark bundle.

Family	Tier	Paths	Comps.	Pages	Roles	Svcs.	Uploads	Tables	Public	Private	Priv. med/max	steps
Travel	A	33	23	3	2	1	1	2	5	18		5/15
Travel	B	41	31	4	2	2	1	3	10	19		6/25
Travel	C	57	36	6	3	4	2	6	10	15		8/34
Travel	D	62	37	6	3	4	2	7	11	17		8/34
Grocery	A	23	30	3	2	1	0	2	5	19		6/17
Grocery	B	39	46	4	2	2	0	2	5	24		5/22
Grocery	C	51	58	5	3	4	0	2	7	24		5.5/29
Grocery	D	56	59	5	3	4	0	3	8	26		6/31
Clinical	A	29	25	3	2	1	0	3	6	17		3/12
Clinical	B	47	34	4	4	2	1	6	9	24		4/21
Clinical	C	92	50	5	5	5	4	12	15	37		5/41
Clinical	D	97	51	5	5	5	4	13	16	39		5/49
Campus	A	27	23	3	2	1	0	3	14	10		5/10
Campus	B	44	27	4	4	0	1	7	12	14		7/18
Campus	C	64	29	5	5	1	2	10	12	17		10/30
Campus	D	84	38	6	6	3	4	11	14	20		6.5/39
Media	A	27	23	3	2	1	0	3	14	10		5/13
Media	B	38	22	4	4	0	1	6	12	14		5.5/18
Media	C	47	28	5	5	0	1	7	24	20		6/21
Media	D	74	40	5	6	2	5	9	20	28		5/50
HomeFix	A	50	18	3	2	1	0	2	14	10		5/12
HomeFix	B	50	24	4	4	1	0	4	16	13		5/12
HomeFix	C	55	34	5	5	1	1	6	17	32		12/20
HomeFix	D	54	42	6	6	2	2	8	18	46		13/29

Table 8: Benchmark instance inventory extracted from the frozen YAML contracts and scenario files.

Family	Tier	KB	Paths	Comps.	Acts.	Pages	Roles	Svcs.	Tbls.	Uploads	Priv.	Med.	Max
Campus	A	19.6	27	23	23	3	2	1	3	0	10	5.0	10
Campus	B	31.5	44	27	27	4	4	0	7	1	14	7.0	18
Campus	C	48.6	64	29	29	5	5	1	10	2	17	10	30
Campus	D	58.4	84	38	38	6	6	3	11	4	20	6.5	39
Clinical	A	23.9	29	25	25	3	2	1	3	0	17	3	12
Clinical	B	40.0	47	34	34	4	4	2	6	1	24	4.0	21
Clinical	C	68.8	92	50	50	5	5	5	12	4	37	5	41
Clinical	D	73.5	97	51	51	5	5	5	13	4	39	5	49
Grocery	A	22.4	23	30	30	3	2	1	2	0	19	6	17
Grocery	B	38.6	39	46	46	4	2	2	2	0	24	5.0	22
Grocery	C	58.0	51	58	58	5	3	4	2	0	24	5.5	29
Grocery	D	63.1	56	59	59	5	3	4	3	0	26	6.0	31
HomeFix	A	16.1	50	18	18	3	2	1	2	0	10	5.0	12
HomeFix	B	19.9	50	24	24	4	4	1	4	0	13	5	12
HomeFix	C	28.5	55	34	34	5	5	1	6	1	32	12.0	20
HomeFix	D	35.7	54	42	42	6	6	2	8	2	46	13.0	29
Media	A	19.4	27	23	23	3	2	1	3	0	10	5.0	13
Media	B	25.8	38	22	22	4	4	0	6	1	14	5.5	18
Media	C	31.9	47	28	28	5	5	0	7	1	20	6.0	21
Media	D	47.4	74	40	40	5	6	2	9	5	28	5.0	50
Travel	A	23.7	33	23	23	3	2	1	2	1	18	5.0	15
Travel	B	30.8	41	31	31	4	2	2	3	1	19	6	25
Travel	C	39.1	57	36	36	6	3	4	6	2	15	8	34
Travel	D	44.3	62	37	37	6	3	4	7	2	17	8	34

Table 9: Raw benchmark instance inventory. *KB* is the public contract file size in kilobytes. *Paths* counts observable state paths. *Comps.* and *Acts.* count declared components and actions. *Priv.*, *Med.*, and *Max* summarize private scenario counts and private scenario step lengths.

persistence, and browser history obligations. The map should be read comparatively rather than as a loss weight.

B.3 Scenario Lengths and Scenario Splits

Figure 5 shows how private scenario lengths vary across complexity tiers. Figure 6 summarizes the public and private scenario split across benchmark instances. Together, these figures document the workload behind the aggregate scores reported in the main paper.

The distribution matters because long horizon failures are only observable when a scenario has enough steps to compose multiple obligations. A four step scenario can reveal a missing control or simple state error. A twenty step scenario can reveal a stale projection after a role switch, a reload after an upload, or a final ledger update after several service backed actions. This is why ConformWeb reports both strict scenario success and step level progress.

B.4 Application Family Summary

Table 10 summarizes the behavioral pressures emphasized by each application family. The families were chosen not for domain coverage alone, but to span distinct forms of behavioral coupling in stateful web applications. A model that succeeds only by recognizing booking or cart patterns should not automatically succeed on registrar overrides, clinical command handoffs, campaign release, or warranty workflows.

B.5 Family and Tier Inventory

Table 11 lists the family and tier cards behind the shared tier schema in Table 1. The entries are short descriptors of public benchmark design rather than hidden tests. Numeric contract statistics and scenario step distributions are summarized earlier in this appendix.

This inventory is the bridge between the abstract tier labels and the actual applications. Tier C does not mean the same user story in every family; it means that the family-specific story now includes regulated or policy-sensitive coupling. Likewise, Tier D is not simply "more buttons". It asks the generated application to preserve post-confirmation or release-stage invariants across multiple public views.

The flow cards also show why ConformWeb is not a screenshot reconstruction benchmark. Many

private compositions cross role, route, and persistence boundaries after the initial page has already rendered. The behavior being scored often appears only after the application looks complete under quick inspection.

C Representative Application Surfaces

Figures 7–12 show representative surfaces for the six application families. The screenshots help readers understand the interface contexts behind the public contracts. They are not visual targets, and generated applications are not scored by screenshot similarity.

The surfaces also illustrate the separation between interface form and behavioral conformance. The travel, grocery, clinical, registrar, campaign, and home service families differ in vocabulary, layout, and workflow organization. The evaluator nevertheless interacts with each application through the same contract based abstraction of actions, observations, and behavioral relations. ConformWeb therefore tests whether declared behavior is preserved under browser interaction, not whether an implementation resembles a reference screenshot.

D Result Breakdowns

Additional result views support the aggregate findings in Section ???. The main paper reports trends by application family, model capability, and complexity tier, while the figures below expose variation across benchmark instances and individual models. These views make the aggregate conclusions easier to inspect. A low full pass rate may reflect broad difficulty across families, or it may be driven by a small number of especially hard instances. The breakdowns below separate these cases.

D.1 Family-level Aggregate Results

Table 13 gives the numeric family-level aggregates behind Figure ??. The table confirms that the conformance gap is not confined to one domain. Media Campaign and Campus Registrar have the highest full-pass rates in this slice, while Travel, Grocery, and HomeFix remain much lower. Even in the relatively stronger families, however, step-level progress is far above full-pass rate, which means many runs advance through substantial portions of the workflow before a later relation fails.

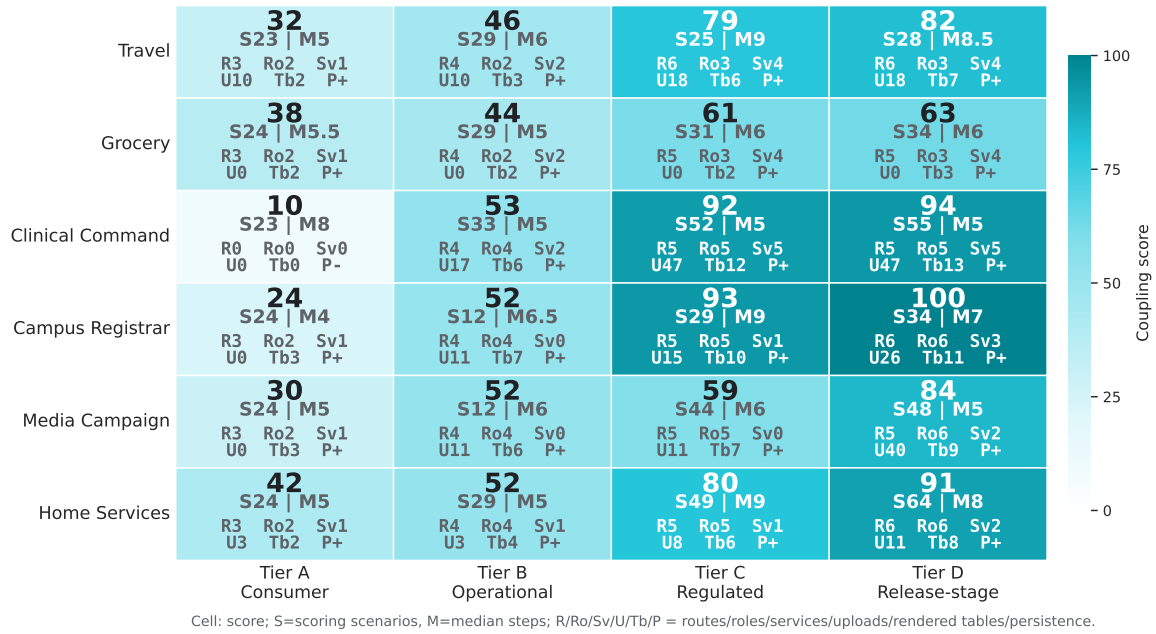


Figure 4: Benchmark complexity map. Each cell summarizes the behavioral pressure of one application family and complexity tier. The summary score is descriptive. Generated applications are evaluated only through private browser scenarios.

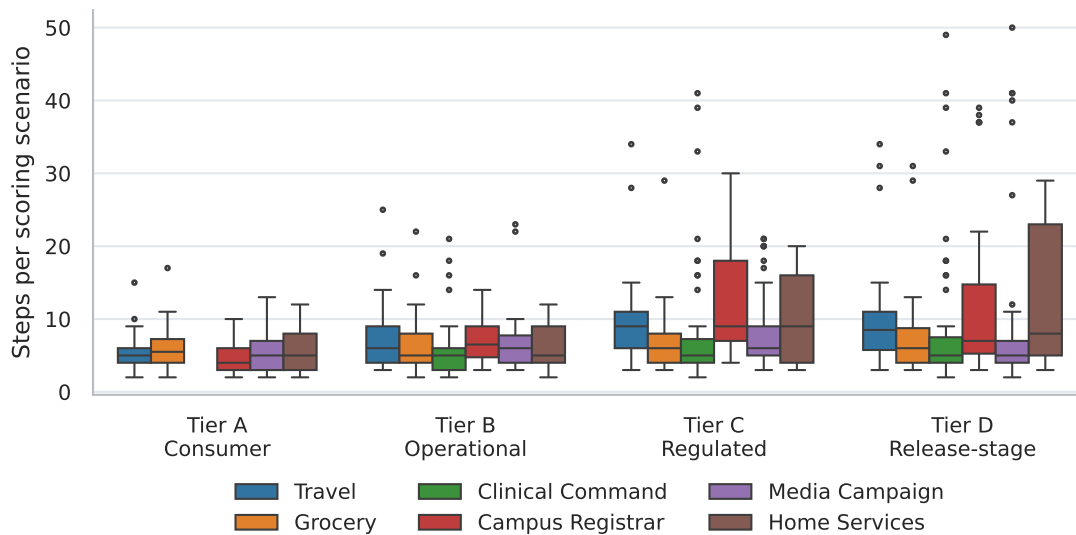


Figure 5: Distribution of scoring-scenario lengths across complexity tiers and product families. Longer scenarios provide more opportunities for late failures and long-horizon drift.

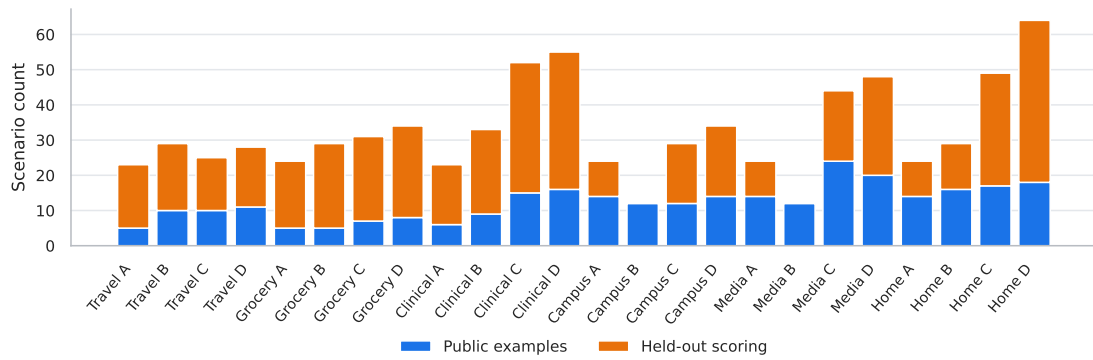


Figure 6: Scenario split across benchmark instances. The split documents how scenarios are allocated among the visibility or evaluation categories present in the data.

Family	Product focus	Representative behavioral pressures
StayFlow Concierge	Travel planning, booking, checkout, concierge flows.	Itinerary state, quote services, add-ons, booking totals, release or confirmation synchronization.
FreshCart Market	Grocery shopping and fulfillment operations.	Cart state, coupons, substitutions, cold-chain review, support tickets, dispatch readiness.
Clinic Shift Command	Clinical operations and shift-command workflows.	Capacity control, authorization, escalation, handoff packets, audit trails, seal or reseal gates.
Campus Registrar Command	Registrar and enrollment workflows.	Enrollment, waitlists, holds, overrides, approvals, release-stage registrar state.
Media Campaign Launch Desk	Campaign launch operations.	Asset review, launch checklists, budget readiness, approvals, release operations, operational ledgers.
HomeFix Hub	Home-service repair and dispatch workflows.	Booking, technician dispatch, parts readiness, quality assurance, insurance, warranty completion.

Table 10: Application family summary. Across families, the shared challenge is preserving behavioral obligations under browser interaction.

Family	Tier A: Consumer	Tier B: Operational	Tier C: Regulated	Tier D: Release-stage
StayFlow Concierge	Booking, itinerary, checkout state	Concierge operations, add-ons, ordered trip items	Supplier/conflict branches, services, persistence	Release or reopen synchronization across itinerary views
FreshCart Market	Cart, coupon, catalog, checkout checks	Fulfillment, substitutions, operational tables	Cold-chain review, uploads, service-backed readiness	Dispatch or support release with synchronized ledgers
Clinic Shift Command	Capacity, queue, authorization basics	Handoff packets, escalation, audit surfaces	Role-gated clearance, incident or seal/reseal checks	Final command release with persistent operational ledgers
Campus Registrar Command	Enrollment, waitlist, hold basics	Registrar operations, overrides, status tables	Policy-sensitive approvals and role-gated actions	Release-stage enrollment or override synchronization
Media Campaign Launch Desk	Asset checklist and readiness basics	Campaign operations, budget and review tables	Approval branches, service-backed budget/readiness checks	Launch or rollback synchronization across campaign ledgers
HomeFix Hub	Quote, booking, timeline basics	Technician dispatch, roster, parts readiness	QA, insurance, invoice, reset or persistence checks	Warranty-command release with dispatch and quality ledgers

Table 11: Family and tier inventory. Each cell summarizes the public focus used to instantiate the shared tier schema. Numeric contract statistics can be reported from the contract inventory without disclosing private test instances.

Family	Tier	Typical public scenario flow	Typical private scoring composition
Travel	A/B	select destination, quote package, update itinerary, checkout state	add-ons, service quotes, reload checks, booking ledger consistency
Travel	C/D	supplier/conflict branches, concierge operations, reopen/release actions	role and release-stage synchronization across itinerary, quote, and ledger views
Grocery	A/B	catalog filter, cart mutation, coupon or checkout validation	substitutions, fulfillment tables, staff transitions, support ticket state
Grocery	C/D	cold-chain review, restricted items, manager approval, dispatch readiness	service-backed compliance, late dispatch/release checks, multi-view order ledger
Clinical	A/B	capacity filtering, patient selection, handoff packet, escalation state	role-gated clearance, audit-trail updates, reload and command-board consistency
Clinical	C/D	insurance/auth roles, incident events, seal or reseal gates	long command journeys combining roles, ledgers, rendered tables, and persistence
Campus	A/B	catalog filtering, schedule planning, transfer upload, advisor actions	role-gated overrides, waitlist state, enrollment and audit table projections
Campus	C/D	policy-sensitive approvals, override branches, release-stage enrollment	synchronized registrar ledgers, browser history, persistence, and final release checks
Media	A/B	asset checklist, readiness filters, budget and review tables	approval branches, service readiness, launch checklist state and projection checks
Media	C/D	budget/readiness services, campaign approval, launch or rollback	release or rollback synchronization across campaign ledgers and operational views
HomeFix	A/B	quote, booking slot, technician or roster selection, parts readiness	dispatch status, QA checks, invoice or visit tables, persistence and route checks
HomeFix	C/D	insurance, reset/reseal, warranty-command release, quality ledgers	long warranty journeys combining homeowner, pro, dispatch, QA, and ledger state

Table 12: Scenario-flow cards by product family and tier group. The table summarizes the kind of interaction trajectories represented in the packaged scenario YAML files without listing every hidden private value.

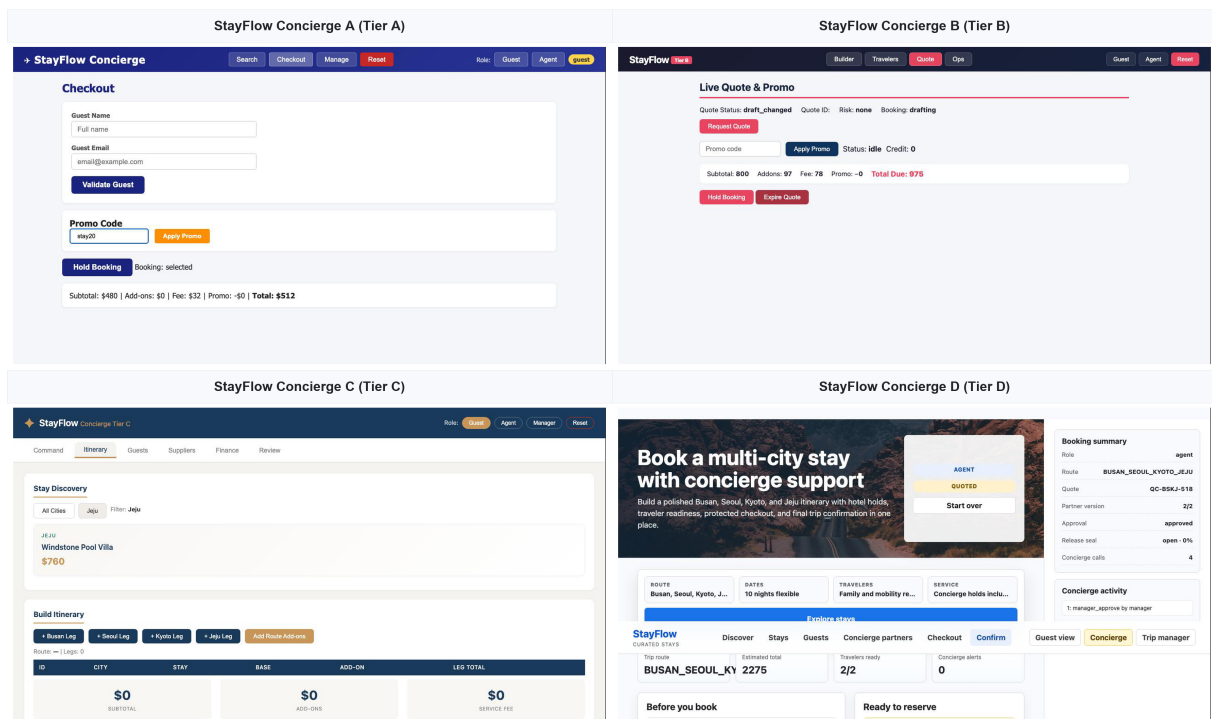


Figure 7: Representative target surfaces for StayFlow Concierge. The screenshots illustrate travel concierge workflows across booking, itinerary, checkout, and add-on coordination. Scoring is based on contract-derived browser interactions, not visual similarity.

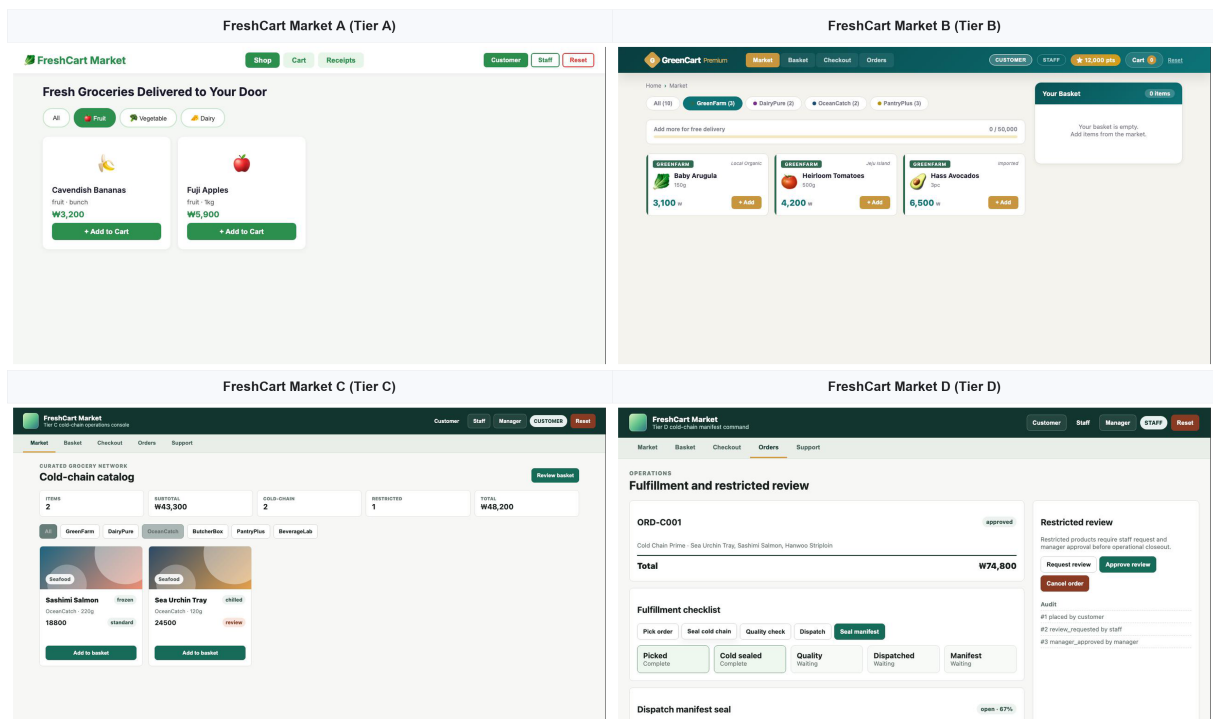


Figure 8: Representative target surfaces for FreshCart Market. The screenshots illustrate grocery fulfillment workflows across carts, coupons, substitutions, and operational readiness. Scoring is based on contract-derived browser interactions, not visual similarity.

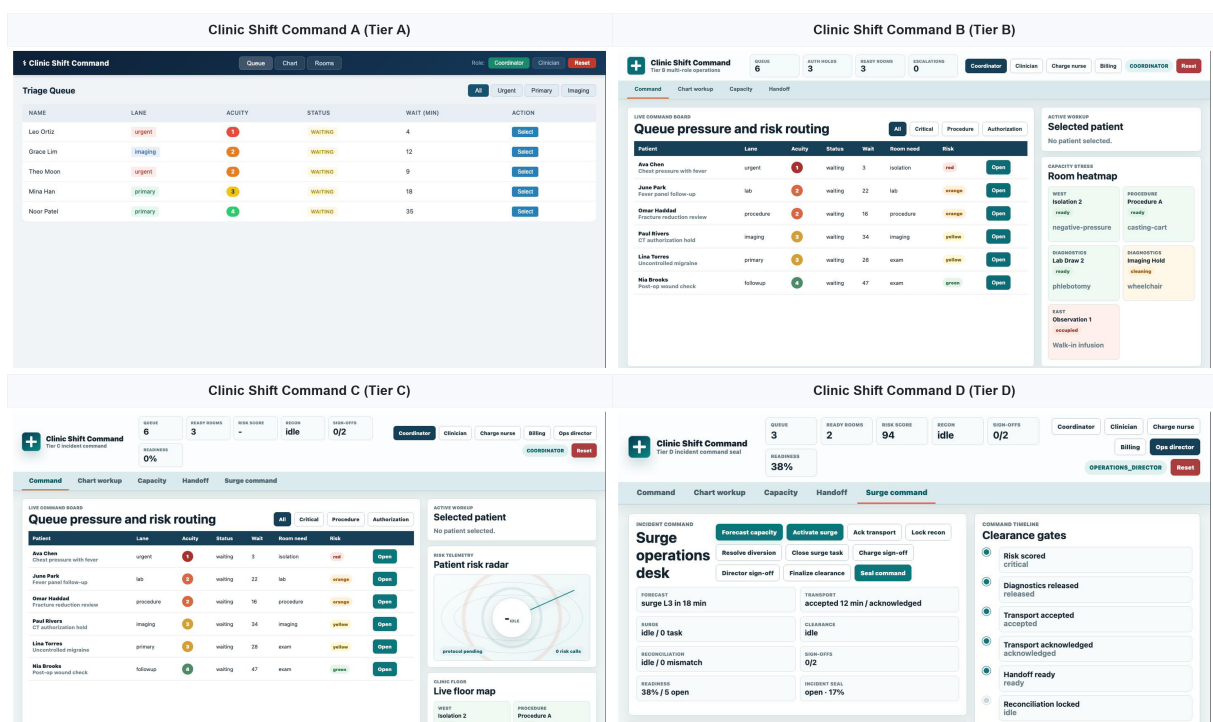


Figure 9: Representative target surfaces for Clinic Shift Command. The screenshots illustrate clinical command workflows across capacity, authorization, escalation, handoff, and audit state. Scoring is based on contract-derived browser interactions, not visual similarity.

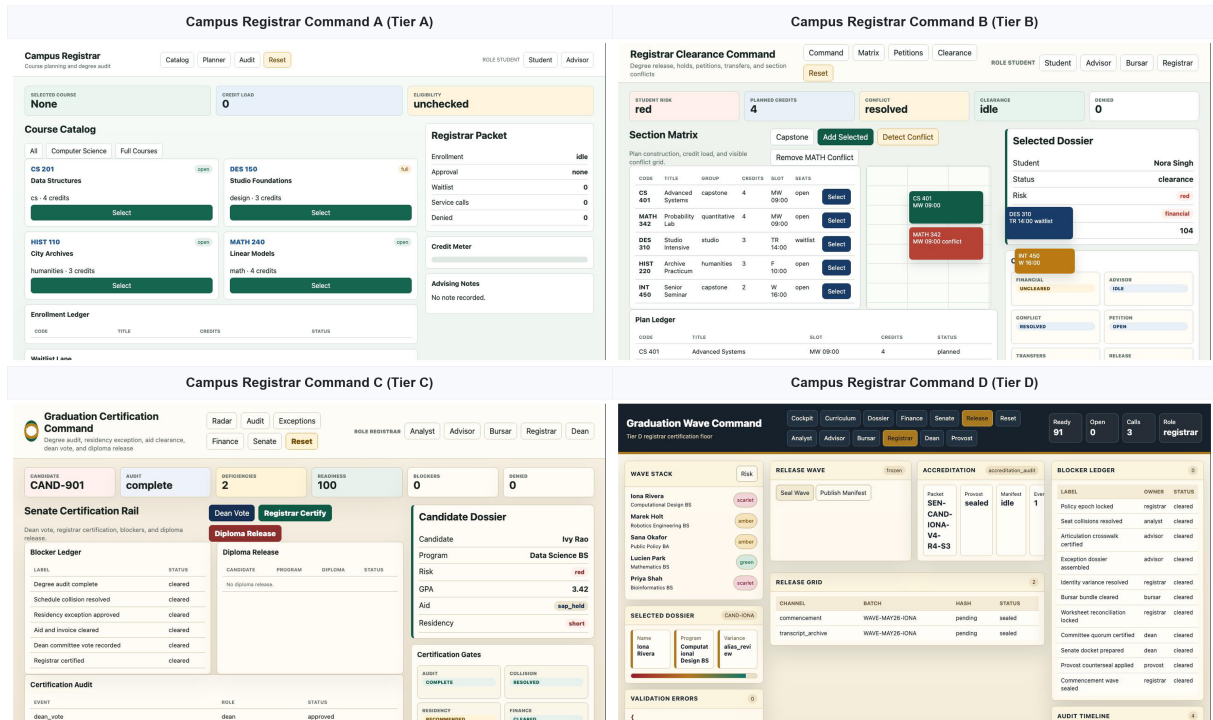


Figure 10: Representative target surfaces for Campus Registrar Command. The screenshots illustrate registrar workflows across enrollment, waitlists, holds, overrides, and approvals. Scoring is based on contract-derived browser interactions, not visual similarity.

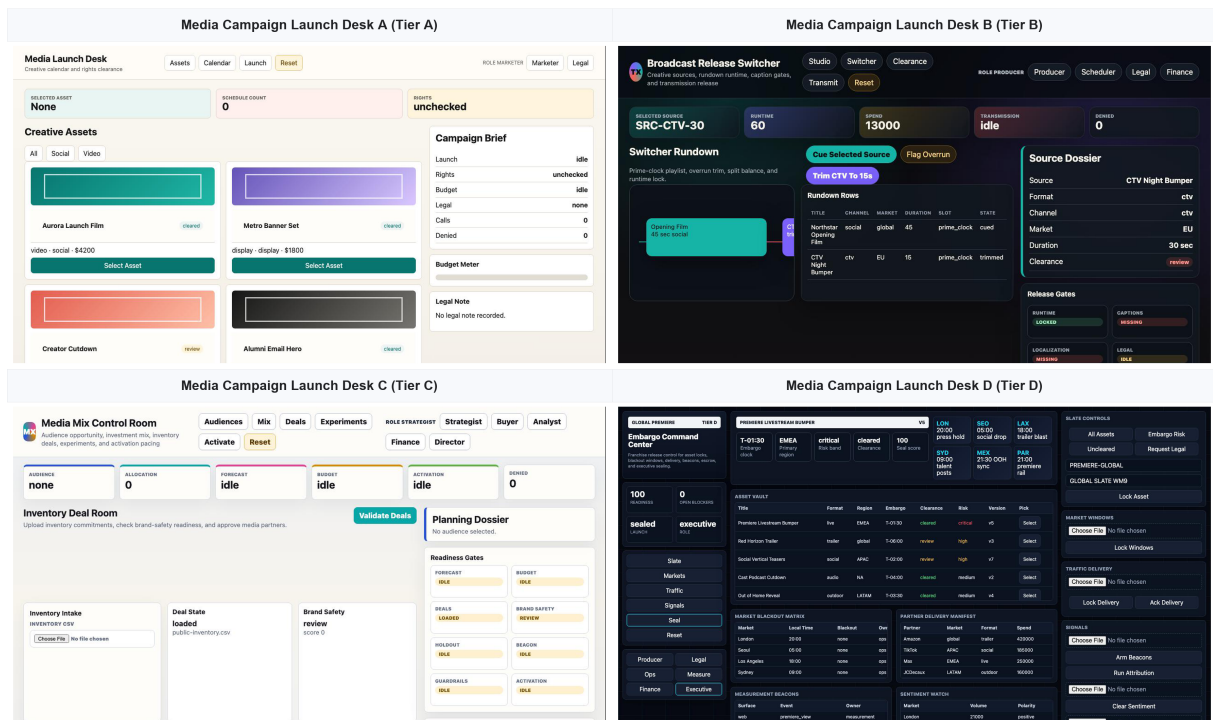


Figure 11: Representative target surfaces for Media Campaign Launch Desk. The screenshots illustrate campaign-launch workflows across assets, readiness checks, budgets, approvals, and release operations. Scoring is based on contract-derived browser interactions, not visual similarity.

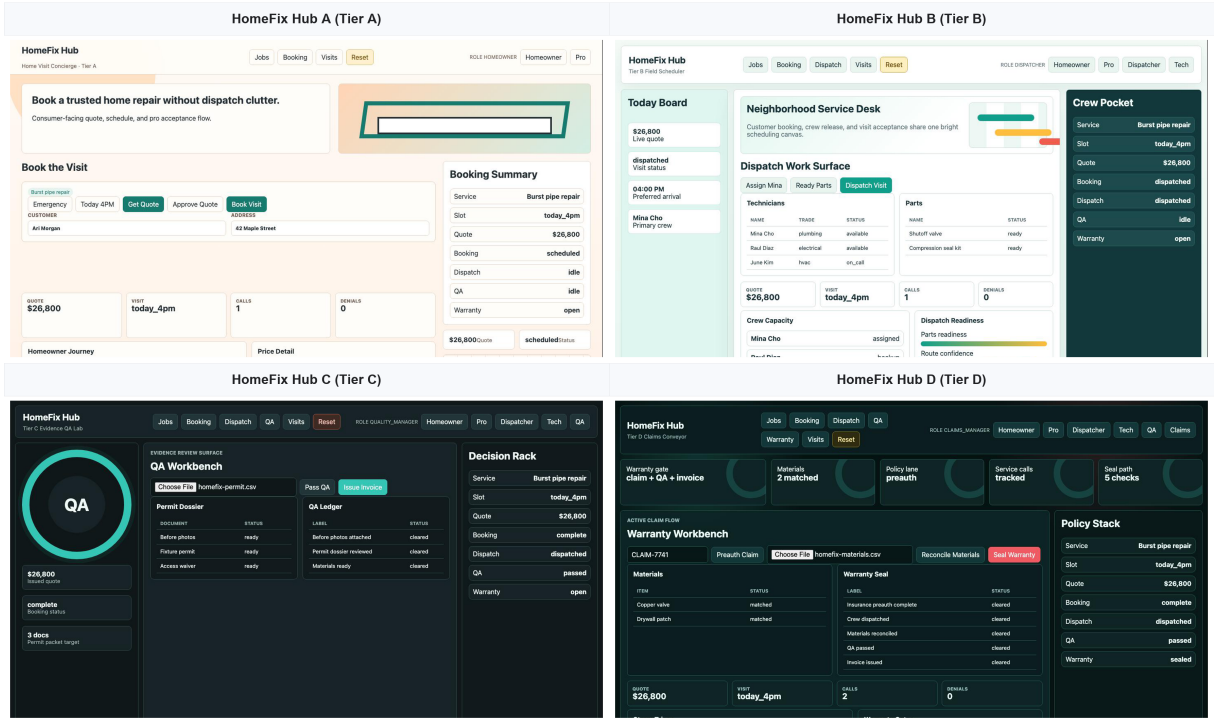


Figure 12: Representative target surfaces for HomeFix Hub. The screenshots illustrate home-service repair workflows across booking, dispatch, parts readiness, quality assurance, insurance, and warranty completion. Scoring is based on contract-derived browser interactions, not visual similarity.

Family	Runs	Full	S_{scen}	S_{step}
Travel	360	6 (1.7%)	28.5	46.0
Grocery	360	20 (5.6%)	24.7	40.2
Clinical	360	43 (11.9%)	42.9	52.9
Campus	360	70 (19.4%)	42.4	52.5
Media	360	86 (24.0%)	45.3	54.2
HomeFix	360	19 (5.3%)	33.8	48.5
Overall	2,160	244 (11.3%)	36.3	49.1

Table 13: Family-level aggregate results. Full-pass counts use generated runs as the denominator; S_{scen} and S_{step} are mean percentages.

D.2 Family-by-tier and Instance-level Results

Figure 13 reports scenario-level conformance for each product family and complexity tier. The heatmap is descriptive: it is not expected to be perfectly monotonic for every family, because later tiers may introduce different behavioral pressures rather than simply increasing length. Figure 14 further breaks results down by model group and benchmark instance.

The instance-level view is important for interpreting model scaling. Frontier models improve nearly everywhere, but the amount of improvement depends on the target. Some low-tier targets remain nontrivial because they require precise state/projection consistency, while some higher-tier targets are

partially solved when the required workflow structure is more regular. The result is a benchmark with heterogeneous failure surfaces rather than a single smooth difficulty ladder.

D.3 Generation Variance

Figure 15 shows variation across independent generations and individual models. This view is useful because a single prompt and contract can yield substantially different implementations across repeated generations. The variance is part of the problem setting: application generation is not only a question of whether a model family can ever produce a conformant artifact, but also whether it does so reliably under independent sampling.

E Failure Analysis Details

The main paper reports first-failure categories at a high level. This appendix adds detailed breakdowns of relation types, first-failure positions, and the large wrong-transition category. The goal is to separate two kinds of evidence: what kind of relation fails, and when in the workflow the first failure appears. Both are needed to understand whether a generated application is missing the surface entirely or drifting after substantial progress.

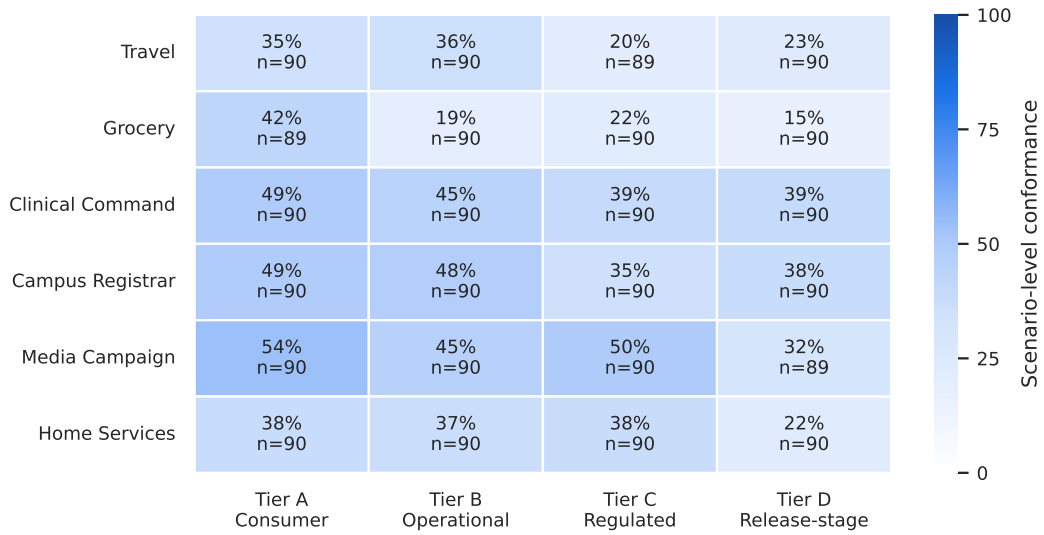


Figure 13: Family-by-tier conformance heatmap. Each cell reports scenario-level conformance for a product family and complexity variant. The heatmap complements the aggregate difficulty-scaling curve in the main paper.

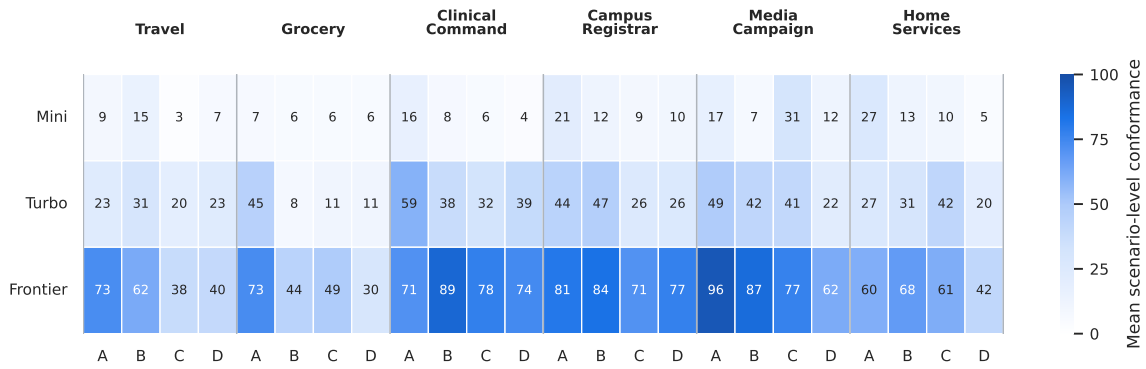


Figure 14: Model-group by benchmark-instance conformance heatmap. This view checks whether aggregate trends are driven by a small number of instances or are visible across the benchmark suite.

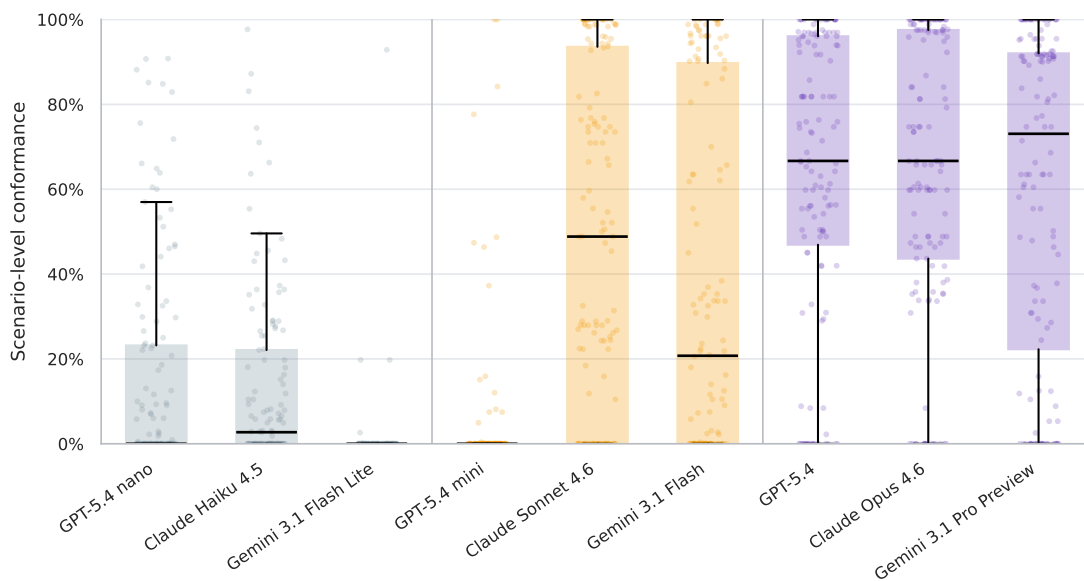


Figure 15: Generation variance across models and capability groups. Independent generations are evaluated separately, exposing the variability of application-level conformance even under a fixed public contract.

E.1 Wrong-transition Breakdown

Wrong state transitions are a broad first-failure category: the action is executable, but the resulting public state violates an expected relation. Figure 16 splits this category into more specific relation-level subtypes when supported by the evaluator metadata. This breakdown is useful because a wrong transition can mean several different engineering failures: a validation map may not update, a service response may be ignored, a role policy may mutate state when it should not, or a ledger may advance to the wrong status.

E.2 Relation-level Failure Breakdown

Figure 17 reports which contract-derived relation types most often appear at the first failing step. This view connects failure categories to the concrete assertions used by the deterministic evaluator. It also makes the failure taxonomy less subjective: the labels used in the main paper are backed by the relation that actually failed in the trace.

E.3 First-failure Position

Figure 18 complements the survival curve in the main paper by showing the distribution of normalized first-failure positions. Immediate failures often indicate missing surfaces or invalid observations, whereas late failures expose state, projection, route, persistence, or release synchronization drift. The timing view explains why step-level progress is a useful secondary metric: two failed scenarios can have the same binary outcome while representing very different levels of implementation fidelity.

F Near-miss Analysis

The main paper uses step-level progress to distinguish immediate failures from near-misses. A near-miss is not a successful implementation: it is a generated application that satisfies many intermediate checks but still violates at least one held-out scenario-level obligation. We include this analysis because near-misses are exactly where visual inspection is most misleading: the application may look polished and may support a long prefix of the workflow, yet still fail the final contract-derived relation.

Figure 19 plots step-level progress against strict scenario-level conformance. Runs in the high-progress but low-conformance region are especially important for ConformWeb: these applications can appear useful during manual inspection, yet fail a

later relation when behaviors are composed. This region is also a useful target for future improvement methods, because it contains runs where the model has learned much of the interface and workflow shape but has not preserved the final invariant.

Figure 20 shows the survival view referenced in Section ?? . Passed scenarios are treated as surviving through the end of the normalized scenario. The separation between model groups should be read as a timing result rather than only an accuracy result: stronger models often fail later, which means they preserve the contract over longer prefixes before drifting.

G Contract Feature Difficulty

Figure 21 reports descriptive associations between contract features and scenario-level conformance. These associations should not be interpreted causally. They are intended to help characterize which visible contract pressures are correlated with lower generated-application conformance under the current suite and model set.

The feature view is useful as a diagnostic map. Features such as rendered tables, uploads, longer scenarios, role policies, browser history, and service fixtures are all mechanisms that require consistency across more than one observable surface. A negative association therefore does not mean that a feature is intrinsically hard in isolation; it means that, in this suite, the feature tends to appear in workflows where more public obligations must be kept synchronized.

H Reference Validation and Positive Controls

Every included benchmark instance has a reference implementation that passes the scoring scenarios before generated applications are evaluated. This positive-control requirement helps separate evaluator or contract errors from model failures. Figure 22 contrasts reference performance with the distribution of generated-application performance.

The reference line should not be read as a leaderboard entry. It is a sanity check for target executability: the contract and private scenarios are not asking for impossible behavior, and the browser evaluator can execute the plan successfully against at least one implementation. Generated applications are still blind samples and do not receive reference source, traces, screenshots, or private failure feedback.

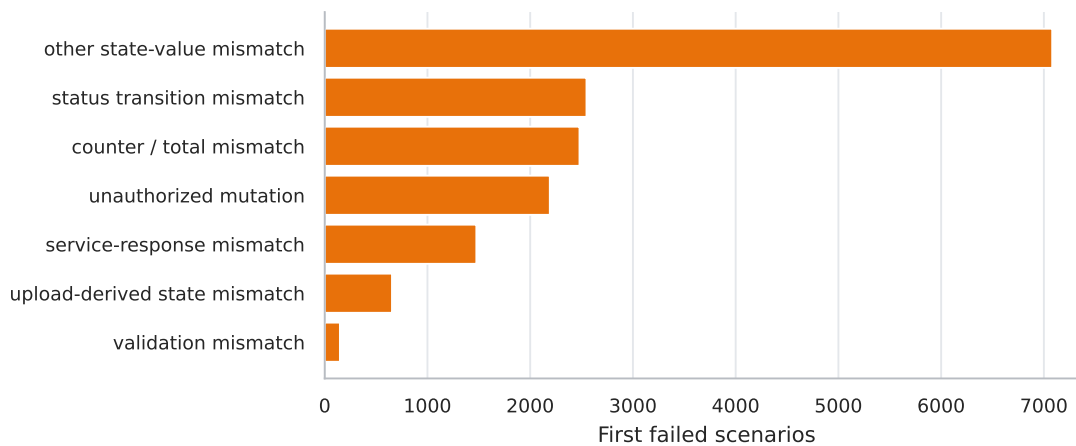


Figure 16: Breakdown of wrong-state-transition failures. The broad category covers actions that execute but produce incorrect public state, such as validation, count, status, service-response, or ledger inconsistencies.

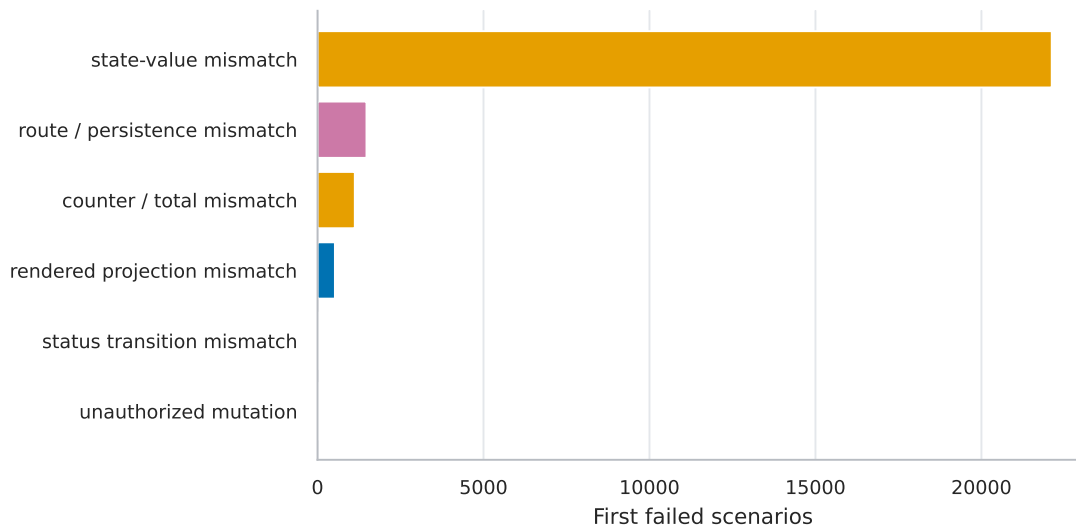


Figure 17: Relation-level breakdown of first failures. The chart reports which rule-based assertions most often identify the first contract violation.

The reference control does not make the benchmark easy. It establishes that the public contract and held-out browser scenarios are executable by at least one implementation. Generated-application failures can then be interpreted as behavioral conformance failures under the disclosed contract rather than as failures of the evaluator to execute the target behavior. Appendix J spells out why this is not iterative repair: failed reference candidates are discarded without feeding private failure information back into the next generation.

I Qualitative Failure Evidence

Figures 23–32 provide a compact set of representative first-failure evidence panels. We include ten panels rather than the full evidence sheet so that

each example remains readable in the appendix. The selected cases cover multiple product families, tiers, and three common late-failure patterns: wrong state transition, route/persistence drift, and rendered-projection mismatch. Each panel shows the last observable state before the failing check, the first state used for the failed contract check, and a short evidence card describing why the pair matters. These screenshots are not used for aesthetic grading; they are audit artifacts collected from the deterministic browser evaluator.

The evidence panels should be read from top to bottom. The top row gives the before/after browser state around the first failing step. The evidence card then states the action, the violated relation, and the expected/observed values when available.

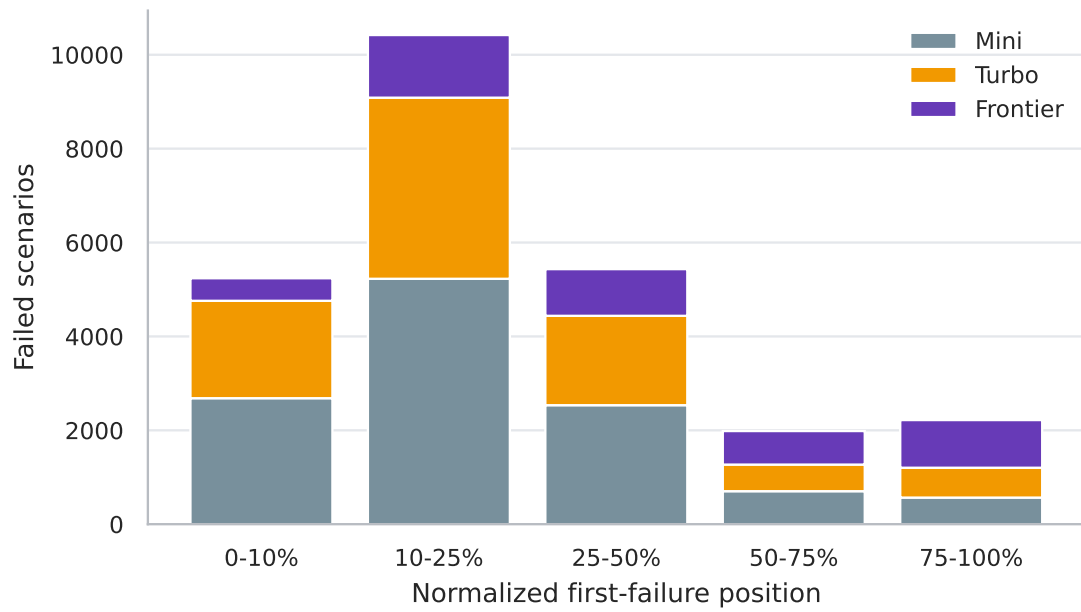


Figure 18: Distribution of normalized first-failure positions. Later first failures indicate implementations that make substantial progress before violating a contract-derived relation.

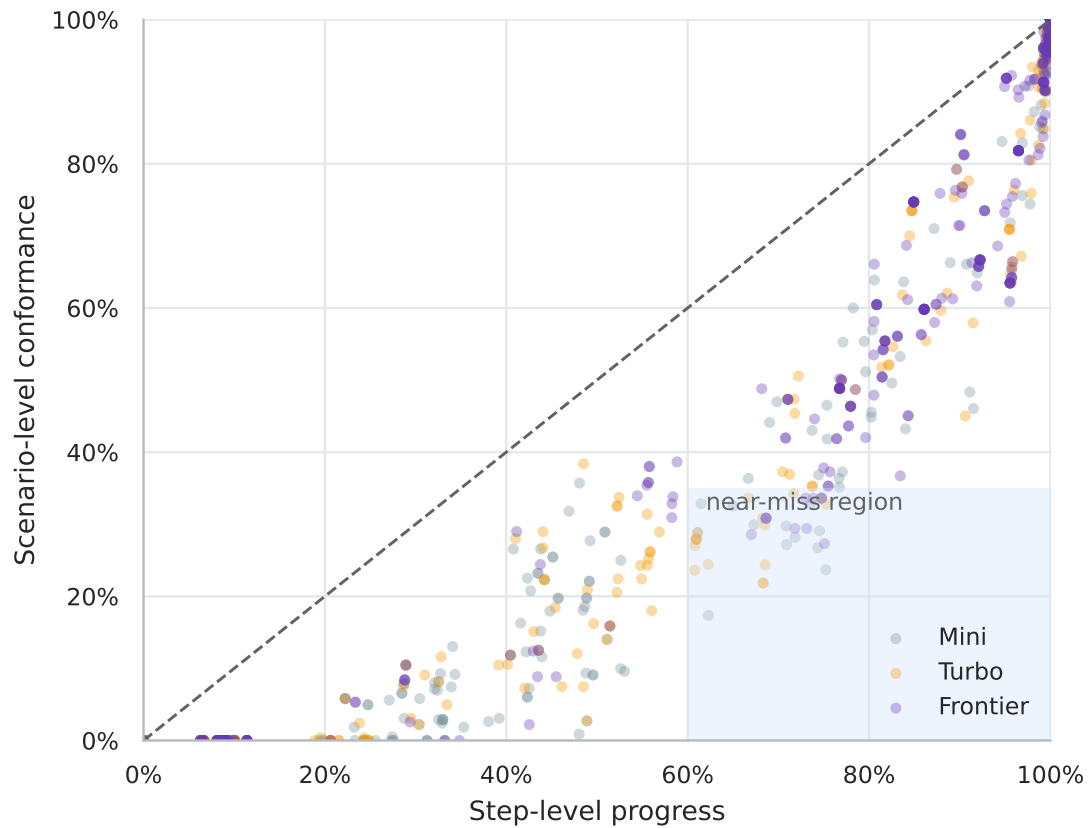


Figure 19: Step-level progress versus strict scenario-level conformance. Runs with high step progress but low scenario conformance are near-misses: they execute much of the workflow but still fail strict end-to-end behavioral conformance.

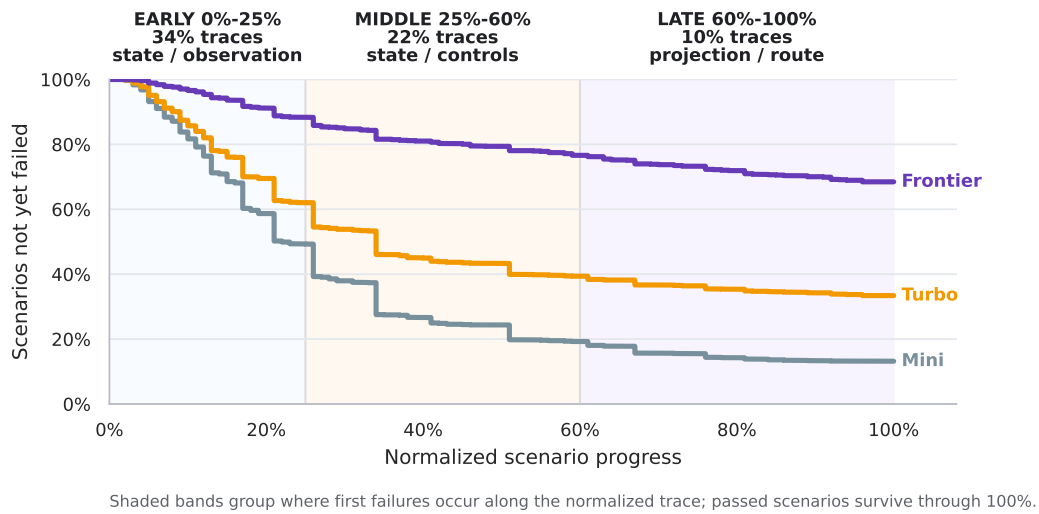


Figure 20: Long-horizon survival of generated applications. Stronger models often survive longer into held-out workflows, but still fail before completing strict end-to-end behavioral conformance.

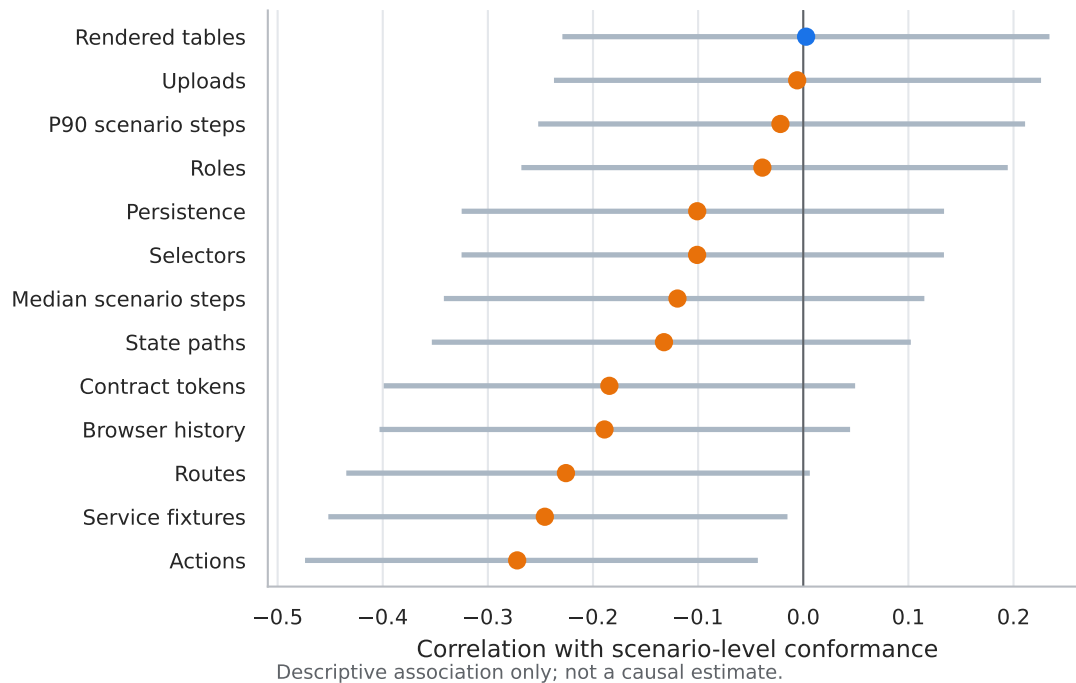


Figure 21: Descriptive association between contract features and scenario-level conformance. Negative values indicate features associated with lower conformance after the analysis controls used by the figure-generation script. These associations are descriptive, not causal.

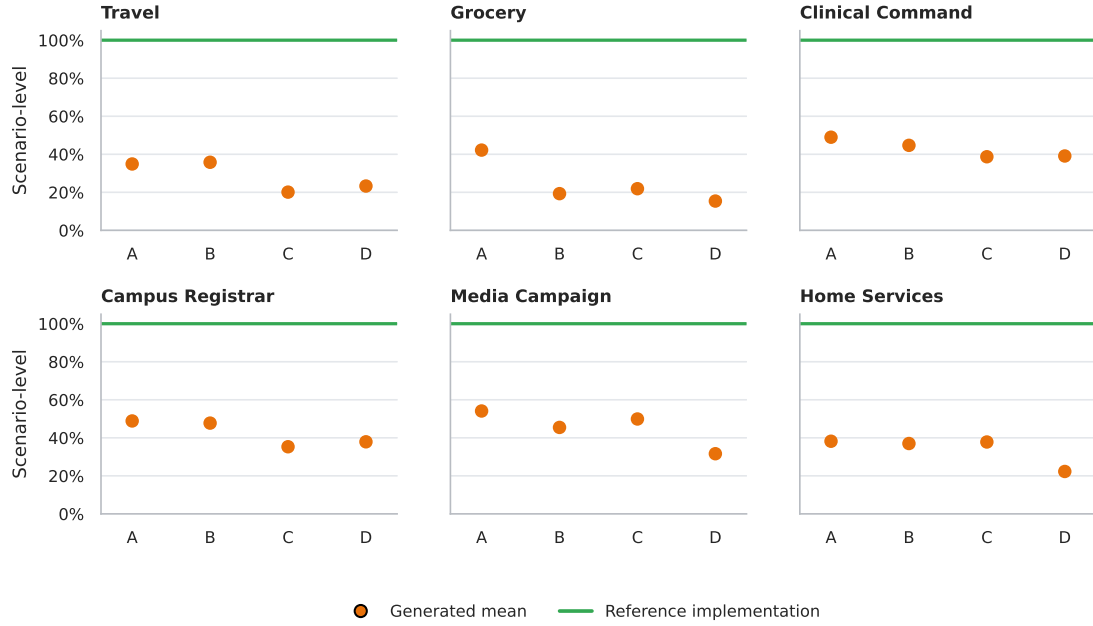


Figure 22: Reference versus generated-application conformance. Reference implementations serve as positive controls for executability and contract completeness; generated applications are evaluated under the same held-out scenarios without seeing reference source, screenshots, traces, or failure feedback.

This format is meant to make failures reproducible and inspectable: a reader can see not only that a run failed, but also the exact point at which a plausible interface stopped satisfying the public contract.

J Reference Generation and Validation Loop Details

The reference implementation is used as an executable witness for a benchmark instance. It is not a source code template and it is not shown to the experimental applications. The goal is to check that the public contract, private scenarios, compiler, and browser evaluator describe behavior that can actually be implemented.

For each application family and complexity tier, the construction process starts from a public behavioral contract, public scenario files when present, private scenario files, and a product premise. The scenarios are compiled into browser actions and checks, then executed by the deterministic evaluator. Candidate reference implementations are built from the public contract and product premise. When a candidate fails, the private scenario files, full execution traces, screenshots, and expected trajectories are not disclosed to the builder. Limited failure summaries are used to identify whether the problem is in the public contract, the compiler, the evaluator, or the reference implementation.

This procedure is therefore a benchmark construction and validation loop, not a model baseline. During construction, failures may lead to corrections in the contract, scenario specification, compiler, evaluator, or reference implementation. Once the instance is frozen, experimental applications are sampled separately under the fixed disclosure boundary. They receive only the public contract and product premise and are evaluated as produced.

Reference construction was deliberately more expensive than the blind generation setting used in the main experiments. In our construction runs, reference applications were developed with Codex using GPT-5.5 as the underlying frontier model. A single application family and complexity tier typically required about five to eight hours of agent assisted iteration before the reference implementation passed the frozen private scenarios. This effort should be read as evidence that the private scenarios are attainable from the public contract, not as evidence that the experimental task is easy.

The distinction matters for interpreting the benchmark. The reference loop answers whether a benchmark instance has at least one implementation that satisfies the private scenarios under the public contract. The experimental evaluation answers a different question. Given only the public contract and product premise, how often do current generators produce such an implementation with-

StayFlow Concierge A (Tier A): Projection mismatch

Scenario: Group manifest upload (Public) · Model: GPT-5.4 (Frontier)

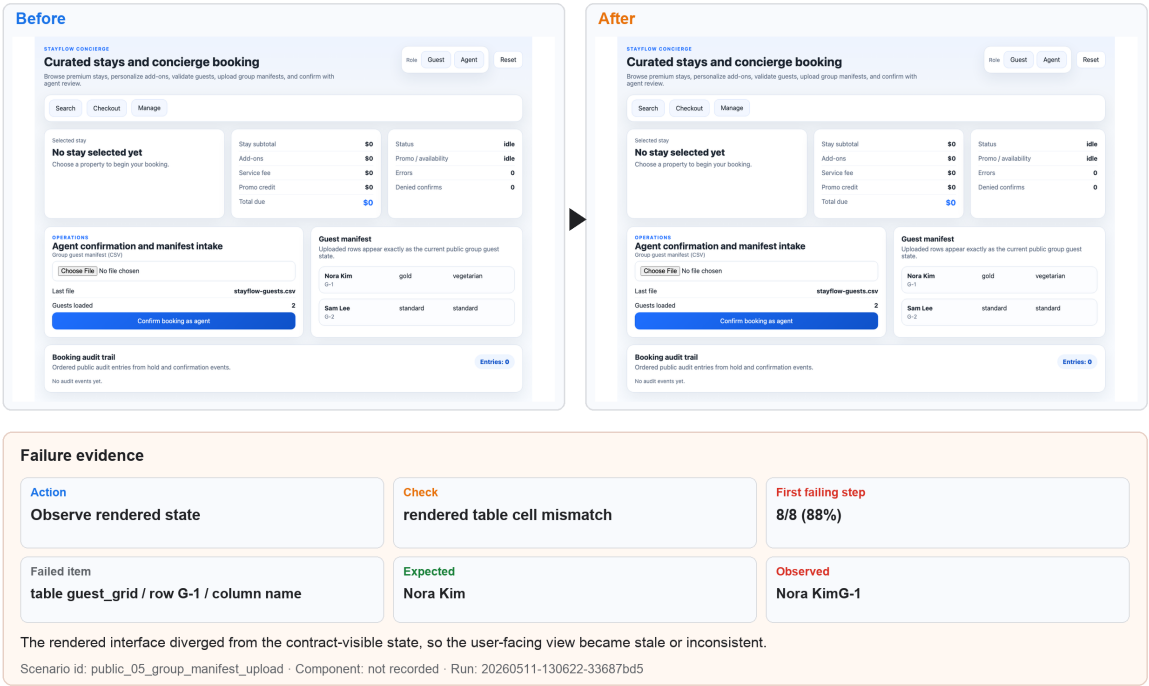


Figure 23: Representative first-failure evidence for StayFlow Concierge Tier A: *rendered-projection mismatch*. The visible booking or itinerary projection no longer agrees with the public state relation checked by the evaluator. Screenshots are audit artifacts collected by the deterministic browser evaluator, not aesthetic judgments.

out construction time iteration or private failure feedback?

Listing 14: Reference validation loop used during benchmark construction.

```
for instance in benchmark_instances:
    while True:
        candidate = build_reference(public_contract,
            product_premise)
        report = run_private_scenarios(candidate)

        if report.passed_all:
            freeze_instance(instance, candidate)
            break

        summary = limited_failure_summary(report)

        if indicates_contract_or_evaluator_defect(summary):
            revise_contract_compiler_or_evaluator(instance,
                summary)
            recompile_private_scenarios(instance)
        else:
            revise_reference_from_public_spec(candidate,
                summary)
```

K Run Accounting and Reproducibility Notes

The completed evaluation set contains 24 validated benchmark instances and 2,157 generated runs. The planned design targets 24 instances, three capability groups, three models per group, and ten independent generations per model. This design

allows up to 2,160 generated runs; the evaluated set contains 2,157 runs after a small number of generation or execution failures.

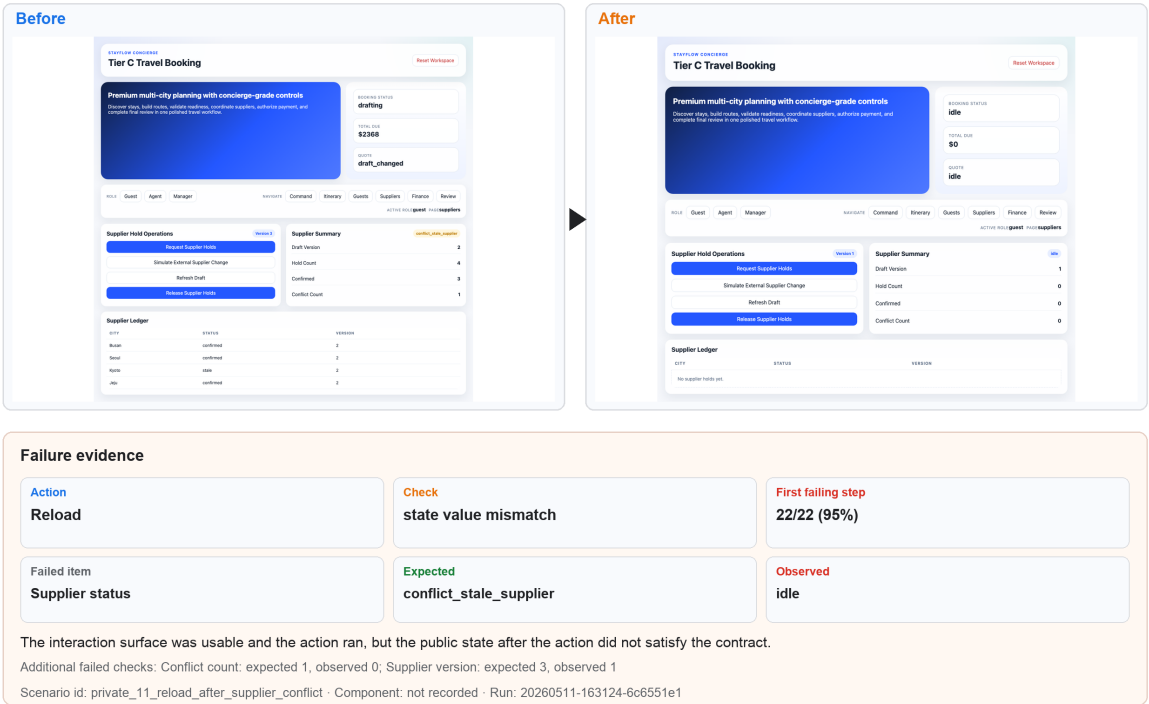
Group	Models	Evaluated runs
Mini	3	719
Turbo	3	720
Frontier	3	718
Total	9	2,157

Table 14: Model-group run accounting for the completed evaluation set. Each benchmark instance targets ten independent generations per model; minor deviations from the planned count are tracked as missing-run notes.

For reproducibility, each generated run should be associated with the benchmark instance identifier, product family, complexity tier, provider-resolved model identifier, prompt hash, contract hash, generation timestamp, build or execution status, and evaluator output. These fields allow the figures in the main paper and appendix to be regenerated from the same run-level and scenario-level logs. They also make excluded runs auditable: a missing generation, build failure, or execution failure can

StayFlow Concierge C (Tier C): Wrong transition

Scenario: Reload after supplier conflict (Private) · Model: GPT-5.4 (Frontier)



FreshCart Market A (Tier A): Wrong transition

Scenario: Save20 coupon credit (Private) · Model: Gemini 3.1 Pro Preview (Frontier)

Before

➔

After

Failure evidence

Action Click Apply coupon button	Check state value outside allowed set	First failing step 12/12 (92%)
Failed item Coupon status	Expected one of ['applied', 'rejected']	Observed idle

The interaction surface was usable and the action ran, but the public state after the action did not satisfy the contract.

Additional failed checks: Last action: expected apply_coupon, observed add_item; Coupon credit: expected 5000, observed 0

Scenario id: private_08_save20_coupon_credit · Component: Apply coupon button · Run: 20260512-040431-2e7ace86

Figure 25: Representative first-failure evidence for FreshCart Market Tier A: *wrong state transition*. A cart or coupon action is executable, but the resulting public state violates the expected count, total, or status relation. Screenshots are audit artifacts collected by the deterministic browser evaluator, not aesthetic judgments.

FreshCart Market C (Tier C): Route / persistence drift

Scenario: Remove item updates counts (Private) · Model: Gemini 3.1 Pro Preview (Frontier)

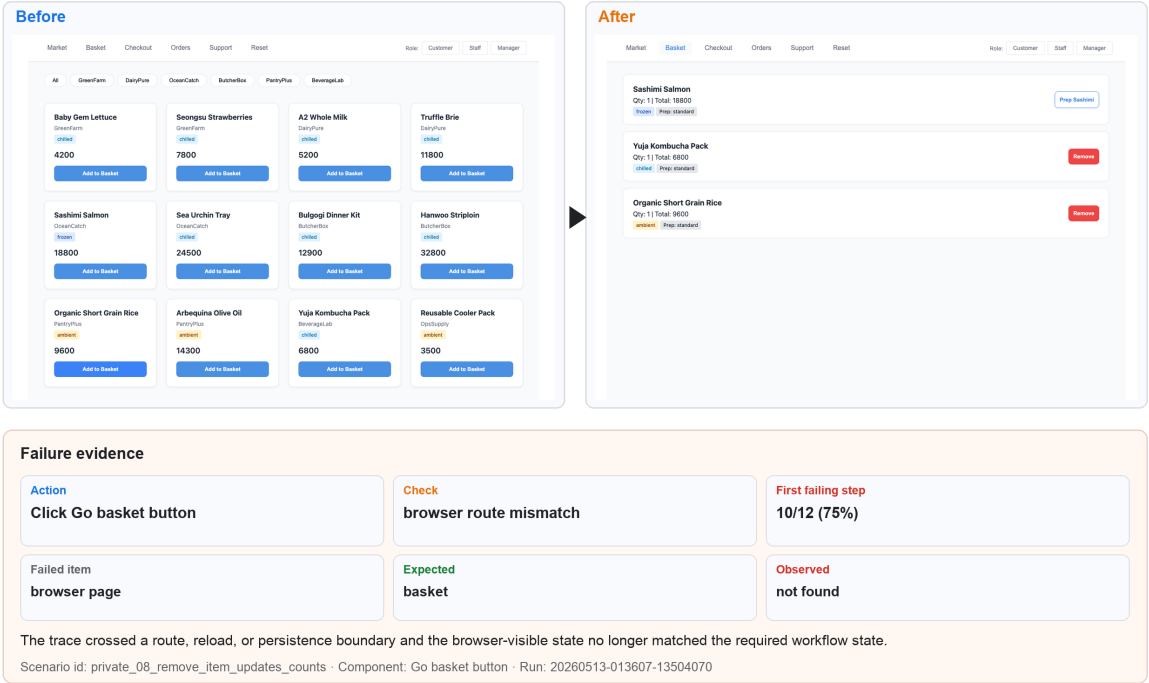
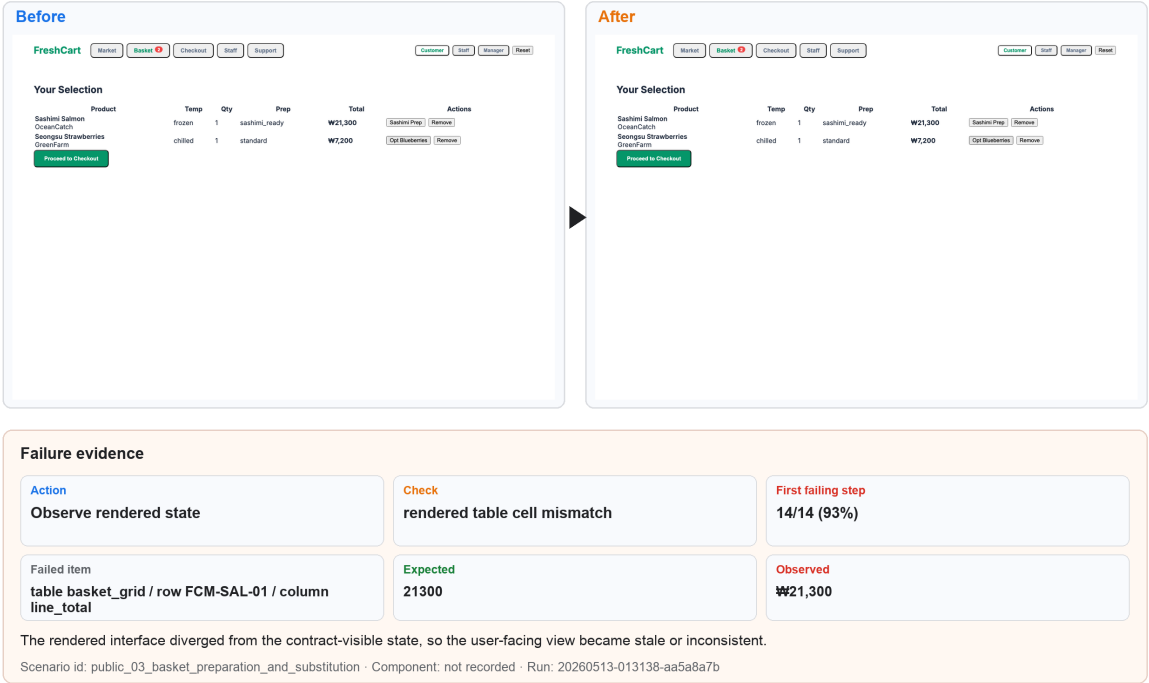


Figure 26: Representative first-failure evidence for FreshCart Market Tier C: *route/persistence drift*. The workflow reaches a navigation or persistence boundary where the required route or state obligation is lost. Screenshots are audit artifacts collected by the deterministic browser evaluator, not aesthetic judgments.

FreshCart Market C (Tier C): Projection mismatch

Scenario: Basket preparation and substitution (Public) · Model: Gemini 3.1 Flash (Turbo)



Failure evidence

Action

Observe rendered state

Check

rendered table cell mismatch

First failing step

14/14 (93%)

Failed item

table basket_grid / row FCM-SAL-01 / column line_total

Expected

21300

Observed

₩21,300

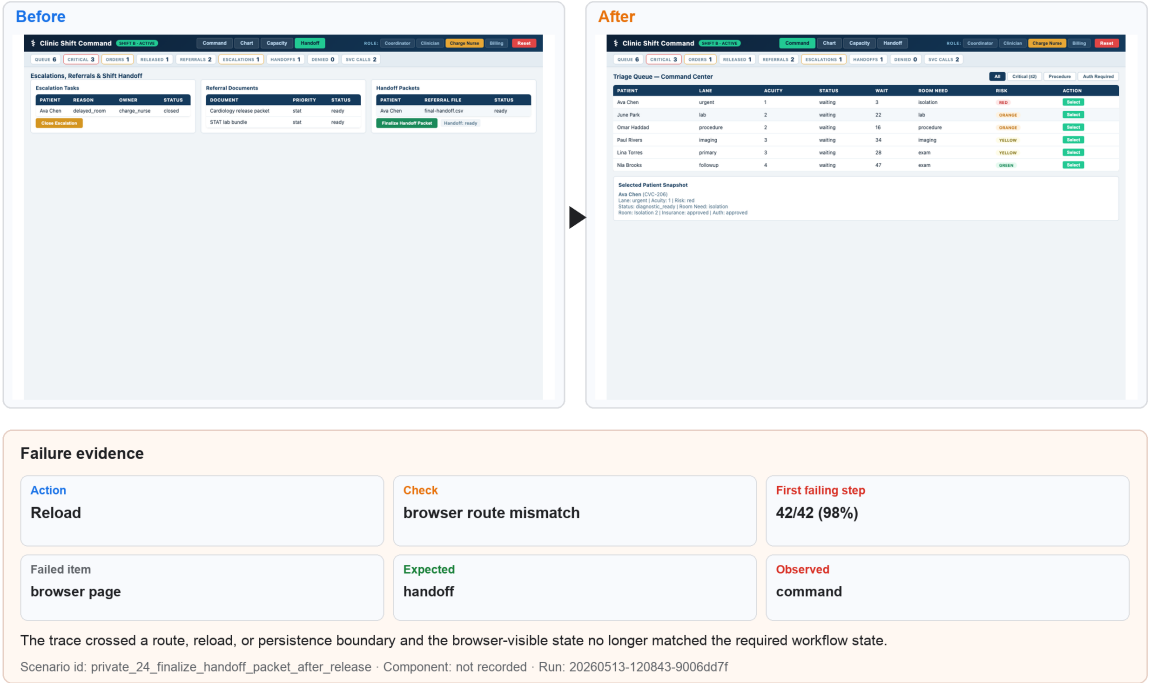
The rendered interface diverged from the contract-visible state, so the user-facing view became stale or inconsistent.

Scenario id: public_03_basket_preparation_and_substitution · Component: not recorded · Run: 20260513-013138-aa5a8a7b

Figure 27: Representative first-failure evidence for FreshCart Market Tier C: *rendered-projection mismatch*. A fulfillment or substitution projection no longer matches the contract-visible state used for the failed check. Screenshots are audit artifacts collected by the deterministic browser evaluator, not aesthetic judgments.

Clinic Shift Command B (Tier B): Route / persistence drift

Scenario: Finalize handoff packet after release (Private) · Model: Claude Opus 4.6 (Frontier)



Clinic Shift Command C (Tier C): Wrong transition
Scenario: Final clearance blocked without signoffs (Private) · Model: Gemini 3.1 Pro Preview (Frontier)

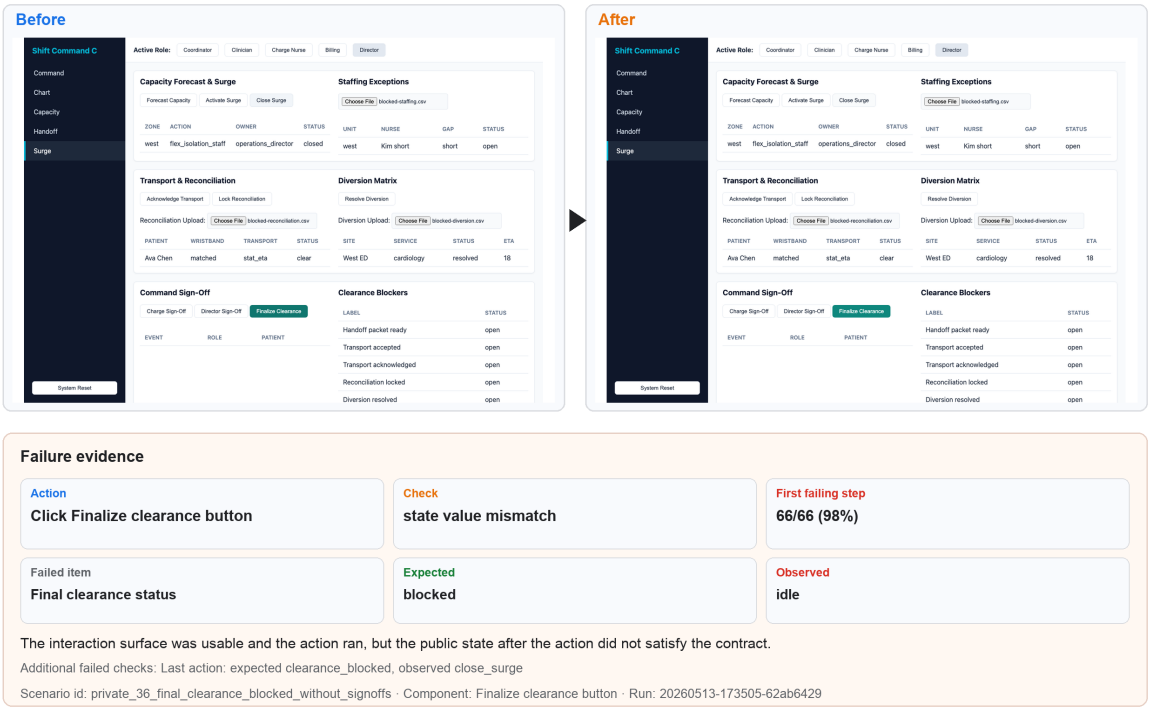
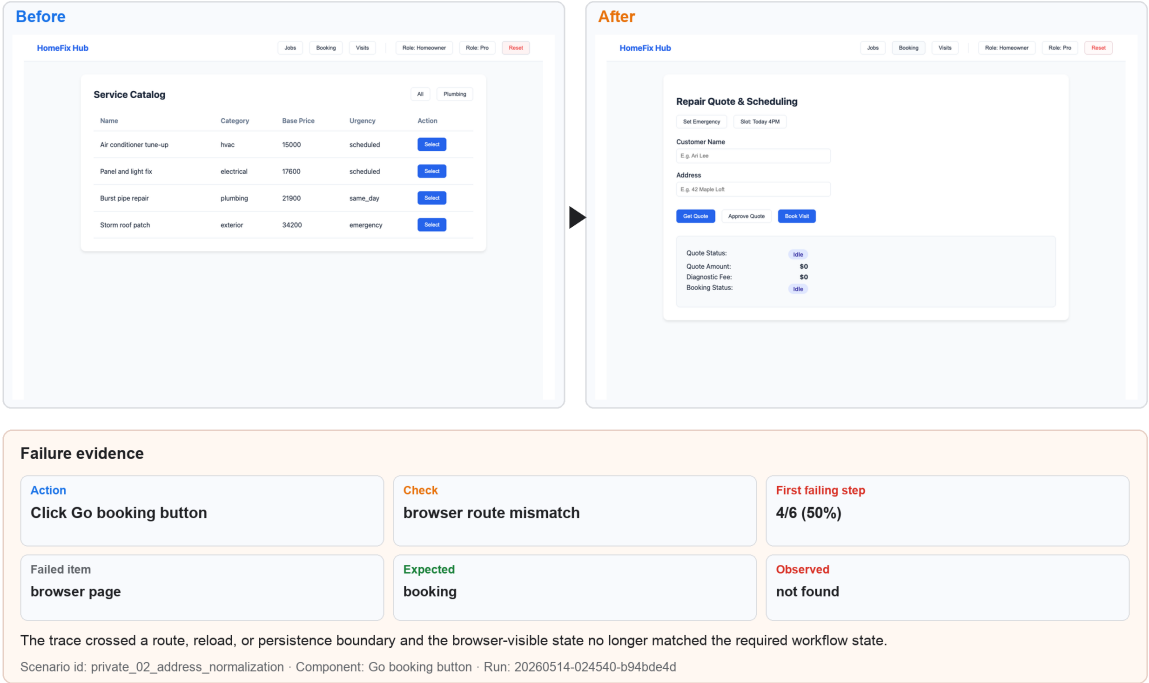


Figure 29: Representative first-failure evidence for Clinic Shift Command Tier C: *wrong state transition*. A clearance or authorization action executes, but the resulting clinical command state violates the expected relation. Screenshots are audit artifacts collected by the deterministic browser evaluator, not aesthetic judgments.

HomeFix Hub A (Tier A): Route / persistence drift

Scenario: Address normalization (Private) · Model: Gemini 3.1 Pro Preview (Frontier)



HomeFix Hub D (Tier D): Wrong transition

Scenario: Dispatch count does not increment without parts (Private) · Model: Gemini 3.1 Pro Preview (Frontier)

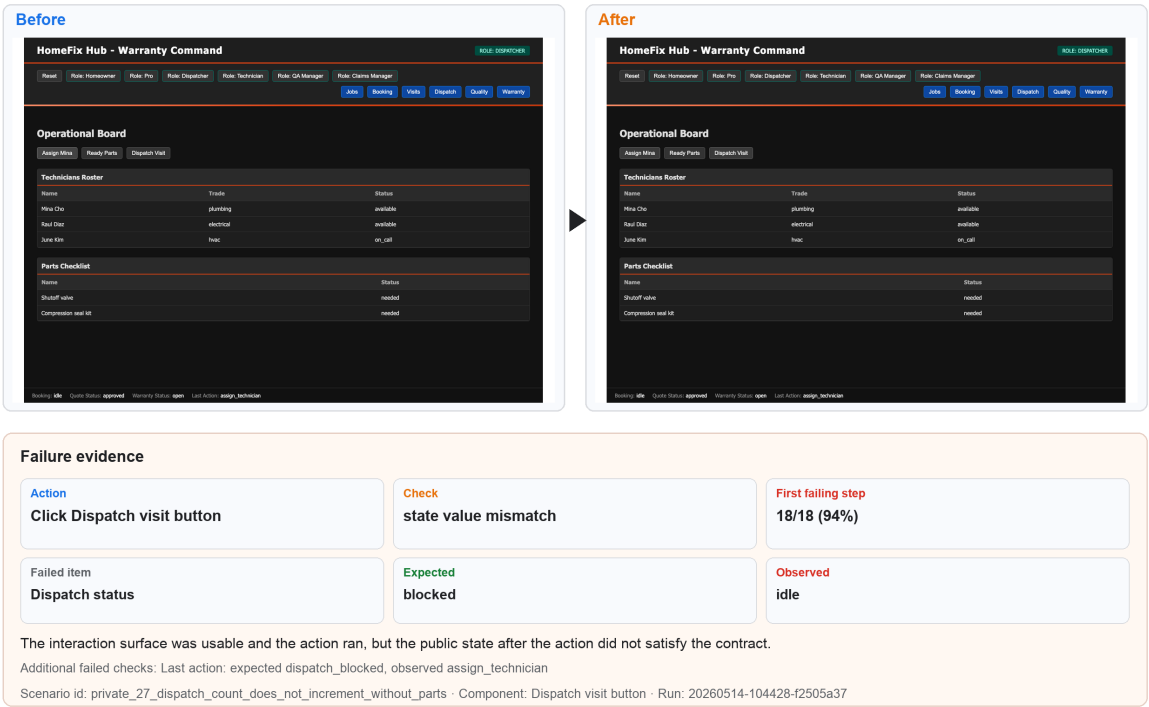
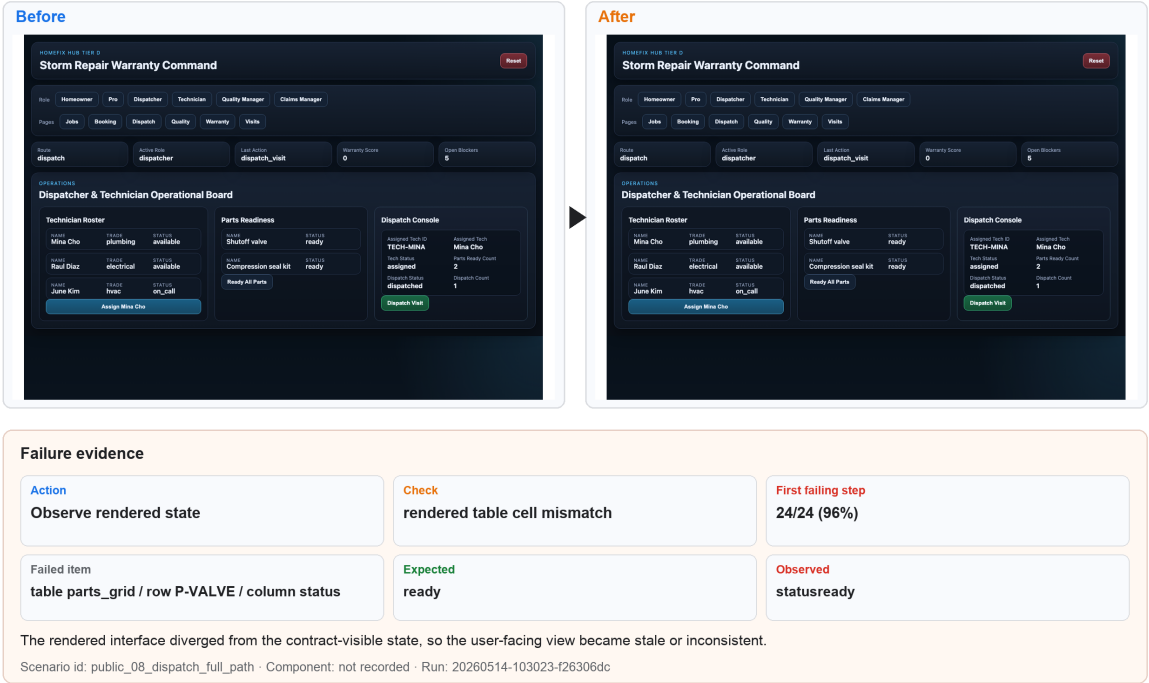


Figure 31: Representative first-failure evidence for HomeFix Hub Tier D: *wrong state transition*. A warranty-command action executes, but dispatch counts, parts state, or operational ledgers fail to update as required. Screenshots are audit artifacts collected by the deterministic browser evaluator, not aesthetic judgments.

HomeFix Hub D (Tier D): Projection mismatch

Scenario: Dispatch full path (Public) · Model: GPT-5.4 (Frontier)



After

HOMEFIX HUB TIER D

Storm Repair Warranty Command

Reset

Role: Homeowner Pro Dispatcher Technician Quality Manager Claims Manager

Pages: Jobs Booking Dispatch Quality Warranty Visits

Route: dispatch

Active Role: dispatcher

Last Action: dispatch_visit

Warranty Score: 0

Open Workbars: 5

OPERATIONS

Dispatcher & Technician Operational Board

TECHNICIAN ROSTER

NAME	TRADE	STATUS
Mina Cho	plumbing	available
Raul Diaz	electrical	available
June Kim	hvac	on_call

Assign Mina Cho

PARTS READINESS

PART	STATUS
Shutoff valve	ready
Compression seal kit	ready

Ready All Parts

DISPATCH CONSOLE

Assigned Tech	Parts Ready Count
TECH-MINA	2
Dispatch Status	dispatched
Dispatch Count	1

Dispatch Visit

Failure evidence

Action

Observe rendered state

Check

rendered table cell mismatch

First failing step

24/24 (96%)

Failed item

table parts_grid / row P-VALVE / column status

Expected

ready

Observed

statusready

The rendered interface diverged from the contract-visible state, so the user-facing view became stale or inconsistent.

Scenario id: public_08_dispatch_full_path · Component: not recorded · Run: 20260514-103023-f26306dc

Figure 32: Representative first-failure evidence for HomeFix Hub Tier D: *rendered-projection mismatch*. The operational projection no longer agrees with the public state after a late-stage warranty or dispatch workflow action. Screenshots are audit artifacts collected by the deterministic browser evaluator, not aesthetic judgments.